



中山大學
SUN YAT-SEN UNIVERSITY

OS 挑战赛 Proj59 (决赛)

SLIM-ARC: 面向内存受限端侧系统的 MoE 推理运行时设计与跨设备评测

队 名: 依托 Agent 答辩 OS
组 员: 欧阳易芃 马福泉 刘昊
指 导 教 师: 赵帅 张献伟
学 校: 中山大学
赛 道: 操作系统功能挑战赛
选 题: Proj 59
日 期: 2026 年 8 月 17 日

目录

1	引言	4
2	背景知识、相关工作与研究动机	5
2.1	背景知识	5
2.1.1	端侧 LLM 推理的内存组成	5
2.1.2	MoE Router 与专家访问	5
2.1.3	Prefill、Decode 与长上下文	5
2.2	相关工作	6
2.2.1	权重卸载与内存层次管理	6
2.2.2	MoE 专家预测、缓存与放置	6
2.2.3	KV 管理与计算优化	6
2.2.4	量化与端侧模型压缩	6
2.3	研究动机	6
3	SLIM-ARC 系统设计	8
3.1	设计目标与基本原则	8
3.2	阶段感知的权重访问控制	9
3.2.1	mmap 按需加载与 CPU repack 控制	9
3.2.2	访问阶段与页建议	9
3.2.3	层感知的有界预取	9
3.3	Router 反馈驱动的专家管理闭环	10
3.3.1	专家候选的来源与优先级	10
3.3.2	generation token 与一次性结算	10
3.3.3	错误专家页的安全回收	11
3.4	专家驻留与替换策略	11
3.4.1	shared expert 与 hot expert	11
3.4.2	压力与预测质量控制	12
3.5	统一内存与 I/O 预算	12
3.6	KV cache 与计算协同	12
3.7	优化之间的层次与相互作用	13
3.8	端侧 Agent Harness 扩展	14
4	系统实现	15
4.1	与 llama.cpp 的增量集成	15
4.2	模型所有权与地址生命周期	16
4.3	并发调度与一次性结算	16
4.4	页范围、预算与错误处理	17
4.5	运行时配置接口	17
4.6	跨硬件实现与适配过程	17
4.7	端侧 Agent Harness 实现	18
4.8	指标、测试与复现实现	18

5	实验评估与结果分析	20
5.1	实验设置	20
5.1.1	研究问题	20
5.1.2	硬件与软件环境	20
5.1.3	模型、工作负载与指标	20
5.2	主实验结果	20
5.2.1	x86/WSL: 初赛核心性能与优化链	20
5.2.2	48.41 GB 模型在 2 GiB no-swap 环境中运行	22
5.2.3	RK3588 3 GiB cgroup 主实验	22
5.2.4	RK3588 动态页建议恢复	23
5.2.5	Raspberry Pi 5 慢存储主实验	24
5.3	消融实验	25
5.3.1	Mac 正式机制矩阵	25
5.3.2	专家预测、门控与预算	26
5.3.3	线程、readahead 与普通权重预取	26
5.3.4	KV、FlashAttention 与量化	26
5.3.5	被拒绝的优化	27
5.4	设备专项与补充实验	27
5.5	综合结果分析	29
6	总结、局限性与未来工作	30
6.1	工作总结	30
6.2	主要实验结论	30
6.3	局限性	30
6.4	未来工作	31
A	完整实验数据与复现信息	32
A.1	证据等级与比较规则	32
A.2	模型与设备合同	32
A.3	初赛 WSL/x86 数据	32
A.4	RK3588 完整实验数据	33
A.4.1	小模型与首次 80B 上机	33
A.4.2	动态 MADV、长上下文与专家机制	34
A.4.3	3 GiB、超长 Prompt、多轮对话与 RSS	34
A.5	Mac/Colima 完整数据	35
A.6	Raspberry Pi 5 完整数据	36
A.7	HiDevLab 与 Ascend 数据	37
A.8	跨设备优化决策	37
A.9	代码、数据与复现入口	38
A.10	赛事声明与成员分工	38

摘 要

混合专家模型 (Mixture-of-Experts, MoE) 以稀疏激活降低单个 token 的计算量, 但端侧部署仍需映射完整权重, 并同时容纳页缓存、KV cache 与运行时缓冲。当模型文件远大于物理内存时, 推理性能由 Router 访问、操作系统换页、存储链路和计算阶段共同决定, 固定预读或单一缓存策略难以跨设备工作。本文提出 SLIM-ARC, 一套面向内存受限端侧系统的 MoE 推理运行时。其核心是由阶段识别、专家观测、预算准入、异步预取、反馈回收和自适应驻留构成的闭环, 而不是对整个模型施加静态内存建议。

SLIM-ARC 从四个层次控制推理的物理内存与 I/O。权重层采用 mmap、关闭 CPU repack, 并依据 Prefill/Decode 阶段选择访问建议; MoE 层使用层局部和跨层 Router 信息预测专家, 以 generation token 对齐预测与实际选择, 对错误预取的完整内部页执行安全回收; 资源层以 cgroup 压力、统一 I/O 预算、预测浪费和专家热度约束准入; KV 与计算层结合低比特 KV、StreamingLLM eviction 和 FlashAttention, 降低长上下文内存与计算开销。运行时由模型对象持有, 图执行通过租约访问, 从而使后台工作、映射地址和析构顺序具有明确边界。系统还提供一个低优先级的端侧 Agent Harness, 通过常驻模型服务、短上下文窗口、输出预算和受限 Shell 控制多轮工具调用的上层开销。

实验覆盖 x86/WSL、RK3588、Raspberry Pi 5、Mac/Colima 和华为 HiDevLab aarch64 环境。初赛 WSL 同合同实验中, SLIM-ARC 在 8 GiB 下将 80B 模型 Decode 从 0.08 提升至 0.43 token/s; 固定 32 GiB warm-cache 合同下, FlashAttention auto 又将 Decode 从 3.01 提升至 5.16 token/s。48.41 GB 的 Qwen3-Next-80B-A3B Q4_K_M 在 Mac/Colima 的 2 GiB、4 vCPU、no-swap 条件下完成推理, 终态复核达到 Prefill 3.908455 token/s 和 Decode 0.709095 token/s。RK3588 的 3 GiB cgroup 实验中, SLIM-ARC 将 Decode 从 0.70 提升至 2.21 token/s; 另一组阶段消融表明, 静态 MADV_RANDOM 会把 80B 推理降至 0.44/0.26 token/s, 而阶段自适应策略恢复至 2.84/1.40 token/s。Raspberry Pi 5 的慢存储实验中, shared/hot 专家驻留相对 demand-paging 参考将 Decode 提高 14.16%、wall time 降低 6.58%; 16 MiB 预算在吞吐近似不变时将投机 weight I/O 降低 98.44%。结果说明, 端侧 MoE 优化必须依据阶段、内存压力、缓存状态和存储链路进行设备化选择, 且不同实验合同的收益不能直接相乘或合并。

关键词: 端侧大语言模型; 混合专家模型; 虚拟内存; 异步预取; 专家驻留; 内存受限推理

项目演示 (P2) <https://www.bilibili.com/video/BV1fXTF6HEAw?p=2>

1 引言

大语言模型的参数规模和上下文长度持续增长，推理所需的权重、KV cache 与临时缓冲已经超过许多嵌入式设备和边缘设备的物理内存。MoE 模型通过 Router 为每个 token 选择少量专家，使激活计算量小于总参数量 [1, 2]。然而，稀疏计算并不会自动形成稀疏的物理内存占用：完整权重仍需存放在本地介质，页缓存以操作系统页面为单位工作，Prefill 与 Decode 又具有不同的访问密度。模型、运行时、内核和存储设备之间的粒度错配，使低内存 MoE 推理成为系统问题，而非仅靠量化能够解决的模型压缩问题。

现有工作分别从权重卸载、异步预取、稀疏激活、KV 管理和异构执行切入。FlexInfer 研究端侧张量卸载、保留和异步数据移动 [3]；PowerInfer-2 以细粒度缓存和预取支撑智能手机推理 [4]；KTransformers 将 MoE 热冷专家分布映射到 CPU/GPU 异构执行 [5]。FlashAttention、PagedAttention 和 StreamingLLM 则从 Attention I/O、KV 分页和长序列窗口控制减少运行时开销 [6–8]。这些工作证明了内存层次管理的价值，但端侧 CPU-only 环境仍存在三个相互耦合的问题：页建议随阶段和存储设备改变方向，Router 预测错误会消耗稀缺 I/O 与页缓存，后台预取又必须与模型映射共享严格的生命周期。

本文研究的问题是：如何在改变模型权重、Router 选择和生成语义的前提下，把 MoE 稀疏激活转化为可控的物理内存与存储访问，并使同一运行时能够在不同端侧设备上选择合适策略？SLIM-ARC 将内存建议视为可撤回、受预算约束的运行时动作。系统先识别 Prefill、Decode 与 MoE Decode 阶段，再依据层级和 Router 信息产生候选预取；实际选择到达后，系统结算命中与浪费，回收不再需要的完整内部页，并在压力和热度允许时保留可复用专家。统一预算连接普通权重、专家与 KV 路径，设备策略则决定哪些动作应当开启、收缩或完全关闭。

本文的主要贡献如下。

- 提出一套阶段感知的端侧权重访问控制方法，将 mmap、CPU repack、页建议、层级预取和存储特性纳入同一设计，并通过 RK3588 的静态负优化与动态恢复实验验证阶段划分的必要性。
- 设计 Router 反馈驱动的 MoE 专家管理闭环。系统以 generation token 关联预测与事实，对错误专家只回收完全位于其张量内部的页面，并以 cgroup 压力、浪费 EWMA、专家热度和统一预算控制驻留。
- 实现模型所有权的运行时、租约化访问、有界工作队列和严格指标契约，使地址生命周期、并发行为、降级路径与每次实验能够被复现和审计。
- 构建覆盖 x86、RK3588、Raspberry Pi 5、Mac/Colima 和华为 aarch64 的实验体系。WSL 主实验给出 8 GiB 同合同 Decode 0.08→0.43 token/s 以及固定 32 GiB 合同下 3.01→5.16 token/s 的 Attention 消融；决赛实验进一步报告 48.41 GB 模型在 2 GiB no-swap 环境中的可运行性、3 GiB RK3588、动态页建议、慢盘专家驻留与 I/O 预算的设备内收益。
- 实现一个与运行时协同的最小端侧 Agent Harness。它通过常驻服务、上下文裁剪、输出预算和受限 Shell 控制多轮工具负载，但不将功能闭环误写为模型 TPS 加速。

全文按照问题、设计、实现与验证展开。第2节介绍背景、相关工作与研究动机；第3节给出 SLIM-ARC 的系统设计；第4节说明工程实现和硬件适配；第5节报告主实验、消融实验与跨设备分析；第6节总结局限性和未来工作。完整实验数据与复现入口收录于附录。

2 背景知识、相关工作与研究动机

2.1 背景知识

2.1.1 端侧 LLM 推理的内存组成

端侧 LLM 推理的内存需求由模型权重、KV cache、计算临时量和操作系统页缓存共同构成。模型权重在生成过程中反复读取，KV cache 随上下文长度增长，临时张量则随算子实现和线程配置变化。对于 48.41 GB 的 Qwen3-Next-80B-A3B Q4_K_M，当物理内存限制为 2-8 GiB 时，不可能让全部权重同时常驻；系统只能依靠文件映射、demand paging 和页缓存不断搬运工作集。Linux cgroup v2 提供 `memory.max`、`memory.current` 与 `memory.peak` 等边界和观测量，可用于构造可复现的物理内存约束 [9]。

llama.cpp 采用 GGUF 权重与 mmap 加载路径，使模型文件无需整体复制到匿名内存 [10]。mmap 只保留虚拟地址映射，首次访问仍可能触发 major page fault 并从存储读取。`posix_madvise` 和 `madvise` 允许运行时向内核表达 SEQUENTIAL、RANDOM、NORMAL、WILLNEED 或 DONTNEED 等访问建议 [11]。这些接口是 best-effort 提示：它们可以改变 readahead 和页缓存行为，却不保证数据何时到达，也不改变模型计算结果。

2.1.2 MoE Router 与专家访问

MoE 层包含 Router、若干 routed experts 和可能存在的 shared expert。Router 根据 token 表征计算专家分数，并选取 top- k 专家执行 [1,2]。Qwen3-Next-80B-A3B 包含数百个 routed experts，而每个 token 只激活其中少量专家 [12]。这使专家权重具有稀疏工作集，但稀疏性只在 Router 结果产生后才成为事实。在结果到达前进行预取必须依赖历史、跨层或在线转移预测，错误预测会带来额外 I/O 和页缓存占用。

专家张量的布局进一步限制了回收粒度。一个专家切片可能不是页大小的整数倍，首尾页也可能与相邻专家共享。对整个切片直接调用 DONTNEED 可能覆盖仍会被访问的边界页。因此，安全回收必须验证张量可整除性、专家 ID、地址加法和页面边界，只操作完全位于候选专家内部的页面。

2.1.3 Prefill、Decode 与长上下文

Prefill 一次处理多个输入 token，通常按层扫描较大的权重和 Attention 工作集；Decode 每步只产生一个或少量 token，权重访问被重复执行，MoE 专家选择更稀疏。两阶段的计算强度、缺页密度和可用于隐藏 I/O 的时间窗口不同。初赛 Qwen3-4B 的 Phase 2c 实验已出现 Prefill 改善而 Decode 回退 15.2-21.6% 的分化，说明单一访问策略不能同时优化两个阶段。

KV cache 是另一条独立增长路径。PagedAttention 通过分页组织 KV 内存 [7]，KVQuant 与 KIVI 研究低比特 KV 表示 [13,14]，StreamingLLM 通过 attention sink 与最近窗口支持稳定的流式推理 [8]。这些方法与权重换页互补：前者降低上下文相关内存，后者控制远大于物理内存的模型权重访问。

2.2 相关工作

2.2.1 权重卸载与内存层次管理

FlexInfer 将端侧推理表述为张量保留、卸载和异步预取问题,通过灵活的内存管理在设备内存不足时维持执行 [3]。PowerInfer 与 PowerInfer-2 利用神经元激活稀疏性、热冷分布和细粒度缓存,将部分权重放置到 CPU、GPU 或闪存并重叠数据移动 [4, 15]。这些系统表明,预取窗口和保留集合必须受内存预算约束。SLIM-ARC 继承这一原则,但目标环境以 CPU-only llama.cpp、POSIX mmap 和低成本边缘存储为主,运行时动作必须能够在没有 GPU kernel 或专用 NVMe 通路时退化。

2.2.2 MoE 专家预测、缓存与放置

MoE-Prism 从模型与系统协同角度讨论可驻留专家集合与内存预算之间的关系 [16]; ProMoE 使用主动缓存降低专家访问成本 [17]; KTransformers 将热冷专家放置到 CPU/GPU 并构建异步流水线 [5]。EcoSpec 从投机解码出发,将新专家的离散加载成本纳入草稿长度决策 [18]。这些工作共同说明,专家数量不是唯一成本,新增专家是否已驻留、是否触发 I/O 以及能否与计算重叠同样重要。

SLIM-ARC 不依赖训练新的 Router,也不修改原始专家选择。系统使用原生 Router 结果作为事实,用历史、跨层和在线转移信号产生可撤回建议。与主要关注“应预取哪些专家”的工作相比,本文同时处理预测错误后的页级回收、压力下的驻留收缩和模型析构期间的地址安全。

2.2.3 KV 管理与计算优化

FlashAttention 通过 I/O-aware tiling 减少 Attention 对高层存储的访问 [6]; StreamingLLM、H2O、SnapKV 和 InfLLM 分别从 attention sink、heavy hitter、提示相关选择和外部上下文记忆控制长序列 KV 工作集 [8, 19–21]。SLIM-ARC 的 KV q4_0、StreamingLLM eviction 和 FlashAttention 路径沿用这些成熟方向,并将它们放入统一资源视图。本文的主要创新仍位于权重与专家页管理; KV 与计算优化用于构成完整可演示系统,而非替代核心贡献。

2.2.4 量化与端侧模型压缩

GPTQ、AWQ 和 SmoothQuant 从权重或激活分布出发降低模型精度位宽 [22–24]。低比特量化直接减少模型文件与 I/O 字节,但不同量化格式在 CPU kernel、精度和解码速度之间存在权衡。SLIM-ARC 以 Q4_K_M 作为决赛主模型,将 IQ4_XS 作为历史探索;后者在小样本 GSM8K 中出现明显精度风险,因而未成为默认方案。量化减少“每次搬多少”,运行时则控制“何时搬、搬哪些、保留多久”。

2.3 研究动机

上述背景与相关工作揭示了四类尚需在端侧 CPU-only 系统中共同解决的问题。

第一,固定页建议无法适配阶段和设备。Prefill 的顺序扫描与 Decode 的稀疏复用需要不同 readahead; NVMe、虚拟磁盘和 FUSE/USB 对随机缺页的代价也不同。RK3588 上静态 RANDOM 的 4–6 倍负优化说明,将某一设备上的最佳建议复制到另一设备会破坏访问合并。

第二,专家预测必须能够被纠正。仅提高预测命中率不足以保证端到端收益。错误专家会竞争页缓存和存储队列,即便预测本身正确,过晚或过大的预取也可能没有覆盖 I/O 延迟。系统需要把预测、成功建议字节、实际 Router 结果和回收动作关联到同一轮次。

第三，内存与 I/O 预算必须统一且可退化。权重预取、专家预取、专家驻留和 KV cache 共享同一物理内存与存储链路。如果各模块独立扩大窗口，局部指标改善可能转化为整体回退。压力读数缺失、算术溢出或建议失败时，运行时还必须退回原始推理路径。

第四，后台优化必须服从模型生命周期。预取 worker、Router hook 与模型映射并发访问同一地址。进程全局裸指针或无界队列会在模型卸载、图执行失败和重复通知下产生悬空地址或状态泄漏。性能机制只有在地址所有权和结算语义明确后才具有可复现性。

图1把这些问题映射到 SLIM-ARC 的设计响应和实验验证。该映射也是后续章节的组织依据：第3节回答“应当如何设计”，第4节回答“如何落实到 llama.cpp 与端侧硬件”，第5节回答“哪些机制有效、在哪些条件下有效”。



图 1: 研究问题、系统设计和实验验证之间的对应关系。

表 1: 研究动机到设计模块的对应关系

研究问题	设计模块	主要验证
阶段与设备差异	阶段控制器、动态 MADV、设备策略	RK3588 静态/动态对照; Mac/Pi weight prefetch 反向结果
预测错误与 I/O 浪费	Router token、错误页回收、专家驻留	Mac A18–A24; Pi A26–A34; 回收与指标单测
资源竞争	统一预算、压力准入、KV/计算协同	RK3588 3 GiB; Mac 2 GiB; Pi 16 MiB 预算
生命周期与并发	model-owned runtime、租约、有界队列	C++/sanitizer 测试、patched 指标、终态运行
上层多轮负载	常驻 Agent Harness、上下文与输出预算	受限 Shell 功能闭环与 JSONL 可观测性

3 SLIM-ARC 系统设计

3.1 设计目标与基本原则

SLIM-ARC 的目标是在模型文件显著大于物理内存时，利用 MoE 的稀疏访问减少不必要的物理页驻留和同步存储访问。系统不修改模型结构和训练参数，也不把内存建议的成功作为推理正确性的前提。所有优化遵循以下原则。

1. 语义透明：权重内容、张量地址、Router top- k 、KV 内容和采样语义保持不变；页建议失败只能引起性能降级。
2. 事实晚于预测：预取是候选动作，原生 Router 结果才是权威事实；预测必须可以被结算、取消和回收。
3. 资源统一：普通权重、专家权重、KV cache 与驻留集合共同竞争物理内存和 I/O，不能分别无限扩大窗口。
4. 阶段与设备相关：Prefill、Decode、NVMe、虚拟磁盘和 FUSE/USB 的最佳策略可能相反，运行时提供机制，设备策略决定配置。
5. 有界与可观测：队列、预算、窗口、驻留集合和指标版本均有明确上界；异常状态不被解释为可用资源。

这些原则将“尽可能多地预取”改写为“在正确时间对有限候选执行可撤回动作”。图2给出总体结构。

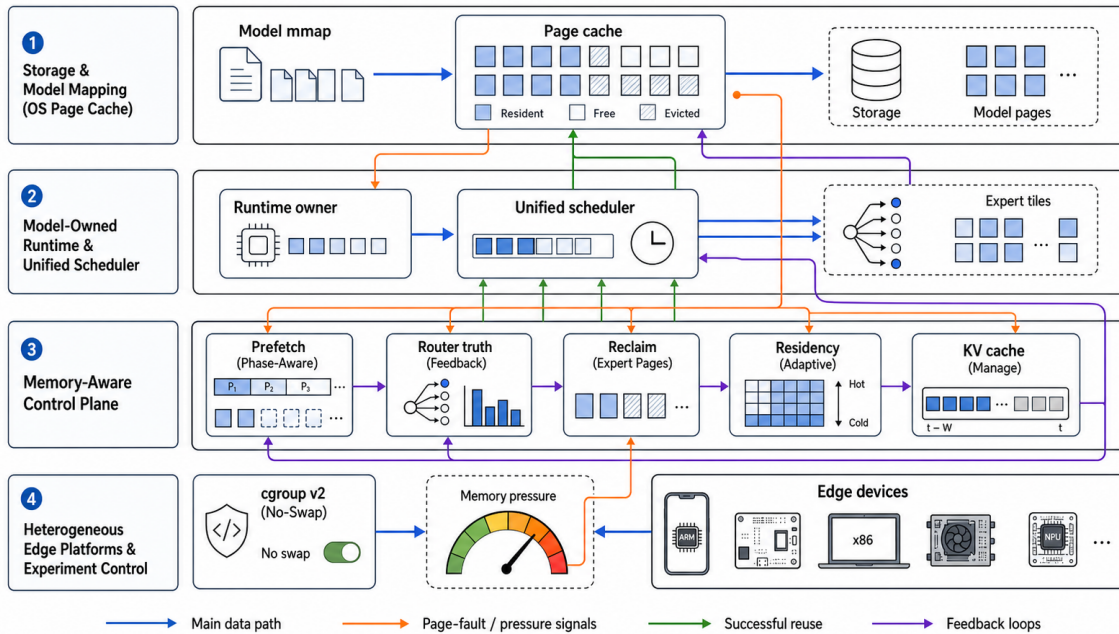


图 2: SLIM-ARC 的系统架构。模型映射、模型所有的运行时、统一调度器和内存感知控制面形成主路径；页错误、压力、Router 事实和复用结果沿反馈路径返回策略层。所有动作通过标准操作系统接口执行，失败只导致性能退化。

3.2 阶段感知的权重访问控制

3.2.1 mmap 按需加载与 CPU repack 控制

模型权重以只读 mmap 映射到进程地址空间。虚拟映射允许 48.41 GB 模型在数 GiB 物理内存下启动，实际驻留由访问和页缓存逐出决定。SLIM-ARC 同时关闭 GGML CPU repack。repack 会为量化权重建立适合特定 kernel 的重排副本，在内存充足时可提高算子效率，但在受限环境中可能使约 45 GB 权重产生接近双倍的物理内存需求。系统保留 GGUF 原始布局，把可用内存留给当前工作集。

3.2.2 访问阶段与页建议

SLIM-ARC 将推理划分为加载、短 Prefill、长 Prefill、短 Decode、长 Decode 和 MoE Decode。Prefill 通常按层扫描较大范围，适合合并顺序读；Decode 的普通权重仍按层重复访问，但 routed experts 稀疏且跨 token 变化。由此形成两级决策：阶段控制器给出默认页建议，设备配置根据存储链路覆盖 Decode 行为。

静态 MADV_RANDOM 会关闭内核 readahead。在随机访问介质上，它可以避免读取未访问页；在 RK3588 SSD 和 FUSE/USB 等链路上，它也可能将顺序读拆成大量同步小请求。因而系统把 SEQUENTIAL、NORMAL 和 RANDOM 视为可选择策略，而不是固定优化。Prefill 默认倾向 SEQUENTIAL；Decode 的普通权重在 Mac/RK3588 可保留适度预读，在 Raspberry Pi 慢盘上则由预算和开关进一步收缩。

3.2.3 层感知的有界预取

设当前计算图的层范围为 $[l_{\min}, l_{\max}]$ ，阶段相关前瞻窗口为 w ，第 l 层已注册权重集合为 T_l 。预取范围为

$$l \in [l_{\min}, \min(l_{\max}, l_{\min} + w)]. \quad (1)$$

系统只为该范围内且通过预算的页区间发出 WILLNEED。窗口之外的权重继续由 demand paging 加载。慢存储模式使用单 worker、window 1、同层去重和 latest-wins：尚未被 worker 领取的旧请求可由更新 generation 替换，已经进入系统调用的请求则不伪装成可取消。

Algorithm 1 阶段感知的有界权重预取

Require: 计算图 G 、阶段 p 、有效预算 B_w

```

1:  $(l_{\min}, l_{\max}) \leftarrow \text{layer\_range}(G)$ 
2:  $l_{\text{end}} \leftarrow \min(l_{\max}, l_{\min} + w_p)$ 
3: for  $l = l_{\min}$  to  $l_{\text{end}}$  do
4:   for all  $r \in \text{merge\_page\_ranges}(T_l)$  do
5:     if  $\text{bytes}(r)$  不超过剩余  $B_w$  then
6:        $\text{advise}(r, \text{WILLNEED}, \text{policy}(p, \text{device}))$ 
7:     else
8:       记录 skipped bytes 并停止扩张
9:     end if
10:  end for
11: end for
```

3.3 Router 反馈驱动的专家管理闭环

3.3.1 专家候选的来源与优先级

SLIM-ARC 使用四类无需训练的候选信号。Temporal predictor 使用上一 token 同层的专家集合；popularity predictor 使用饱和热度计数；inline Router 在原生 Router 执行后立即发布当前层事实，缩短 cache 更新延迟；cross-layer Router 复用下一层 gate 权重，在当前层 expert FFN 和下一层 Attention 期间提前计算下一层 top- k 。候选按照稳定性和时效性排序，并由预算截断，而非简单取并集。

Temporal predictor 成本最低，但对输入变化敏感。Cross-layer Router 能在 Mac 上达到更高预测命中率，却增加一次 gate matmul，因此高命中不保证 wall time 最优。系统保留多种候选来源，设备策略可关闭边际收益不足的预测。在 Raspberry Pi 慢盘上，最终策略直接关闭 speculative expert prefetch，仅保留确定复用的 shared/hot 专家。

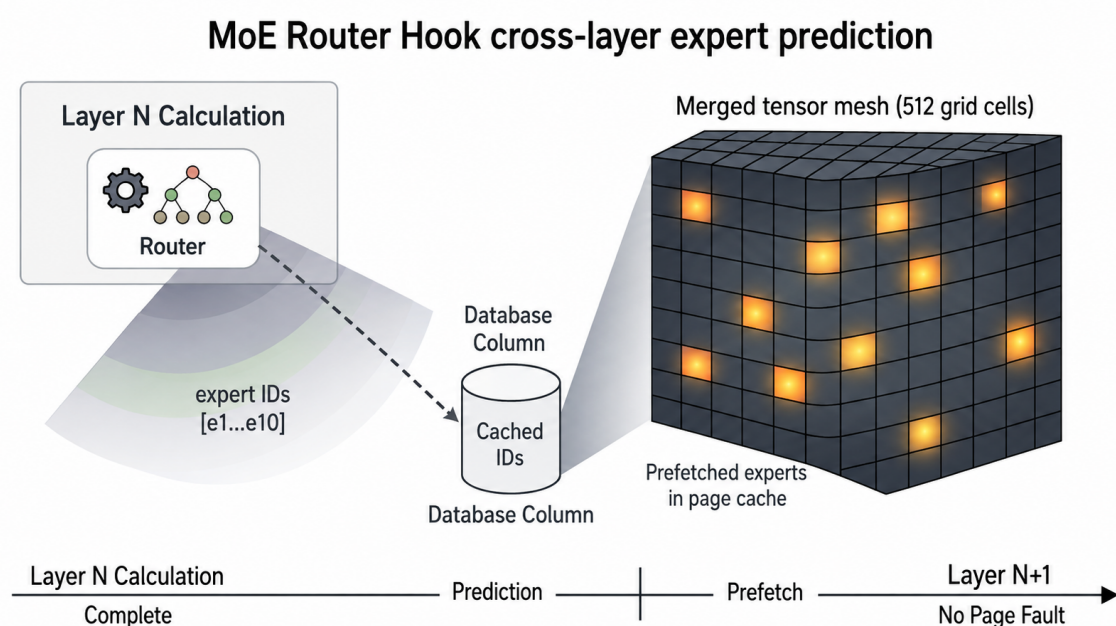


图 3: Router 观测与跨层专家预测。图中缓存 ID 和发光网格表示候选专家页；原生下一层 Router 仍给出最终选择。

3.3.2 generation token 与一次性结算

同一层可以在一个计算图中重复出现，后台 worker 也可能晚于下一次 Router 观测完成。仅以层号保存“最近预测”会把两个轮次覆盖或错配。SLIM-ARC 为每次专家预取分配单调递增的 generation token，并在 advice 前预留有界 pending 记录。记录包含层号、候选专家、每个专家成功建议的字节和状态。

实际 Router 结果只结算相同 token 的记录。成功计算且获得非空 ID 时，系统计算命中和浪费；计算失败、缺少 Router 节点或过滤后为空时，系统显式取消 token，并把已经成功建议的字节归入浪费。每个 token 只能结算一次，token 0 只用于不参与归因的兼容预测。该设计使 Router occurrences、专家集合和成功 advice bytes 使用同一统计单位。

3.3.3 错误专家页的安全回收

令预取专家集合为 P ，实际选择集合为 S ，错误候选为 $W = P \setminus S$ 。对每个 $e \in W$ ，规划器在所有专家张量中定位该专家切片。若张量大小不能被专家数整除、专家 ID 越界、地址加法溢出或页大小无效，则跳过该候选并记录原因。合法切片 $[a, b)$ 被向内对齐为

$$[\lceil a/P_s \rceil P_s, \lfloor b/P_s \rfloor P_s), \quad (2)$$

其中 P_s 为系统页大小。只有非空的完整内部页区间才接收 DONTNEED 建议。这样既不触及选中专家，也不回收与相邻专家共享的边界页。

回收不是推理成功的前置条件。系统调用失败只增加 failure 计数，成功回收字节只统计返回码为 0 的调用。状态锁只用于生成不可变快照，页面建议在锁外执行，避免慢系统调用阻塞 Router hook。

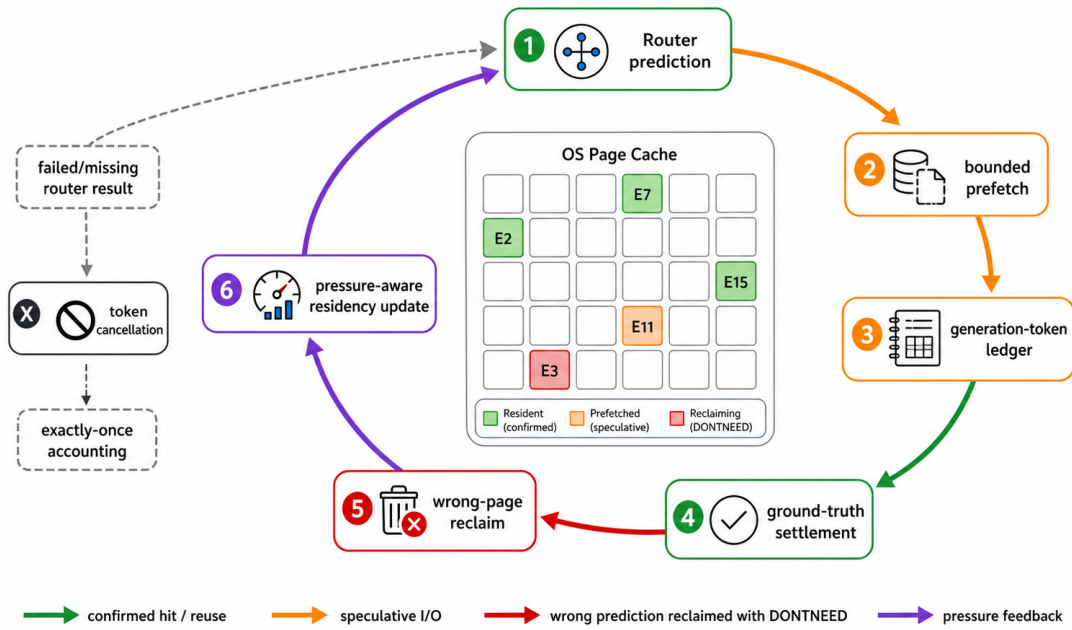


图 4: 专家预测、事实结算、错误页回收和驻留更新构成的闭环。generation token 把预测与原生 Router 事实一一关联；命中页进入复用与驻留更新，错误页只回收完整内部页，失败或缺失的 Router 结果通过取消路径完成一次性记账。

3.4 专家驻留与替换策略

3.4.1 shared expert 与 hot expert

shared expert 在每个 token 上执行，具有确定复用价值；routed experts 的价值由跨 token 访问频率决定。系统先为 shared expert 保留一个较小的锁页集合，再以设备相关容量维护 hot expert cache。驻留不是复制权重，而是对 mmap 页进行 mlock 或保留建议，因此张量地址和 GGUF 内容保持不变。

热度计数使用 uint32 饱和累加，避免长期运行溢出；每 64 个有效 Router 样本统一衰减，防止早期热点永久占据缓存。Raspberry Pi 的实验表明，512 MiB 跨 token LRU 能同时利用频率和最近性；更复杂的 LFRU 虽提高部分 hit counter，却增加非驻留扫描和 wall time。系统因此优先选择能够直接减少缺页和淘汰的简单策略。

3.4.2 压力与预测质量控制

压力控制器以 $r = \text{memory.current}/\text{memory.max}$ 生成 normal、high、critical 或 missing 状态。 $r \geq 0.85$ 进入 high, $r \geq 0.95$ 进入 critical; 恢复使用滞回和连续样本, 避免阈值附近震荡。invalid、zero max、current 大于 max 或无限上限统一进入 missing, 并清除恢复计数。

预测浪费使用千分比样本 x_t 的 EWMA:

$$E_t = \left\lfloor \frac{3E_{t-1} + x_t}{4} \right\rfloor. \quad (3)$$

$E_t \geq 600$ 时收缩预测驻留, 连续两个样本低于 400 后恢复。critical 压力下不准入 routed expert; high 压力下只保留稳定候选; normal 时按 stable、temporal、hot 的顺序选择。预算或最大专家数是硬上限, 重复和非法 ID 被确定性过滤。

3.5 统一内存与 I/O 预算

统一调度器先从静态周期额度 B_{static} 出发。压力准入开启时, 依据 cgroup 上限 M 、当前占用 C 和保留量 R 得到

$$B_{eff} = \min(B_{static}, \max(0, M - C - R)). \quad (4)$$

异常读数采用保守 fallback, 不把算术溢出或缺失状态解释为剩余容量。阶段比例 $\alpha_p = (\alpha_w, \alpha_{kv}, \alpha_e)$ 决定普通权重、KV 与专家路径的 admission 上限:

$$(B_w, B_{kv}, B_e) = B_{eff} \alpha_p. \quad (5)$$

表 2: 统一预算的阶段初始比例

阶段	权重 α_w	KV α_{kv}	专家 α_e
PREFILL_SHORT	60%	10%	30%
PREFILL_LONG	50%	20%	30%
DECODE_SHORT	70%	20%	10%
DECODE_LONG	30%	60%	10%
MOE_DECODE	20%	20%	60%

当前权重和专家路径执行字节级 admission; KV 路径完成策略计算与长上下文模块配置, 但尚未与 model-owned runtime 建立同等强度的逐周期硬限制。因此表2表示初始策略, 而不是三路实际 I/O 带宽的测量值。Raspberry Pi 的 16 MiB 预算和零投机模式进一步表明, 慢存储上的最优额度可能接近零: 预算的价值是拒绝无收益动作, 而非始终填满链路。

3.6 KV cache 与计算协同

权重页管理解决模型文件远大于内存的问题, KV 管理解决工作集随上下文增长的问题。SLIM-ARC 支持 q4_0 KV, 将 key/value 从高精度表示压缩为低比特缓存; StreamingLLM 保留 attention sink 和最近窗口, 将 KV 内存从 $O(L)$ 控制为 $O(S + W)$; FlashAttention 在后端支持时减少 Attention 中间量和高层 I/O [6, 8]。

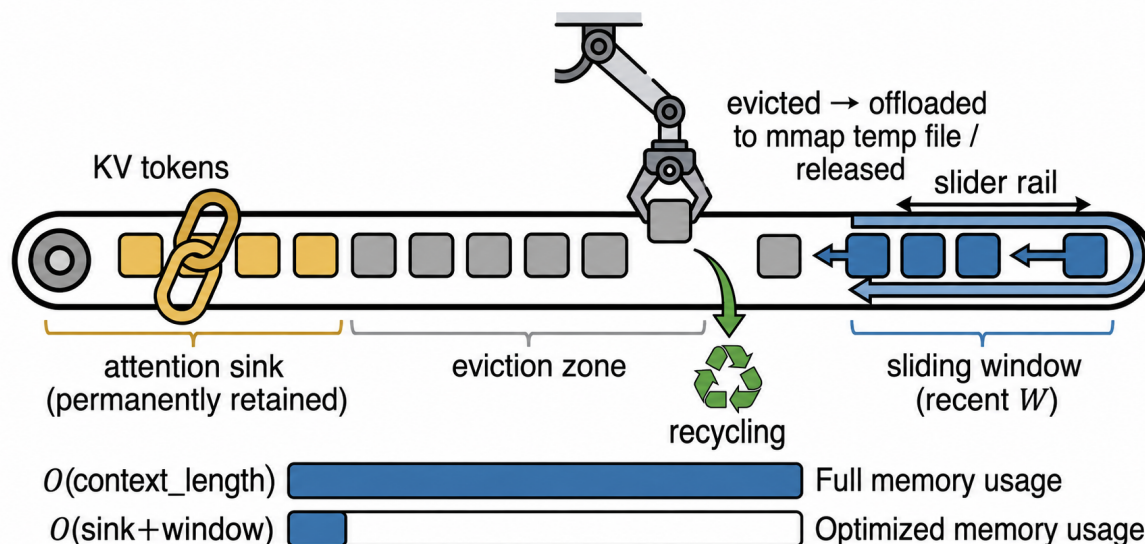


图 5: StreamingLLM KV eviction 的设计示意。attention sink 永久保留，最近窗口沿序列滑动，中间 KV 被释放或卸载。

这些模块不能无条件叠加。顺序删除 KV 可能破坏某些 FlashAttention 布局，低比特 KV 在短上下文小模型上也可能因量化开销变慢。系统按上下文长度、可用内存和后端能力启用相应路径，并在第5节分别报告性能与精度边界。

3.7 优化之间的层次与相互作用

SLIM-ARC 将优化分为释放资源、降低需求和缩短执行三类。第一类包括 mmap、关闭 repack、阶段页建议和错误页回收；第二类包括权重量化、KV 量化、KV eviction、预算和驻留集合；第三类包括 FlashAttention、线程配置、readahead 和能覆盖 I/O 的预取窗口。资源释放能够为缓存和计算优化创造空间，但也可能改变 cache 状态，使简单相加失效。

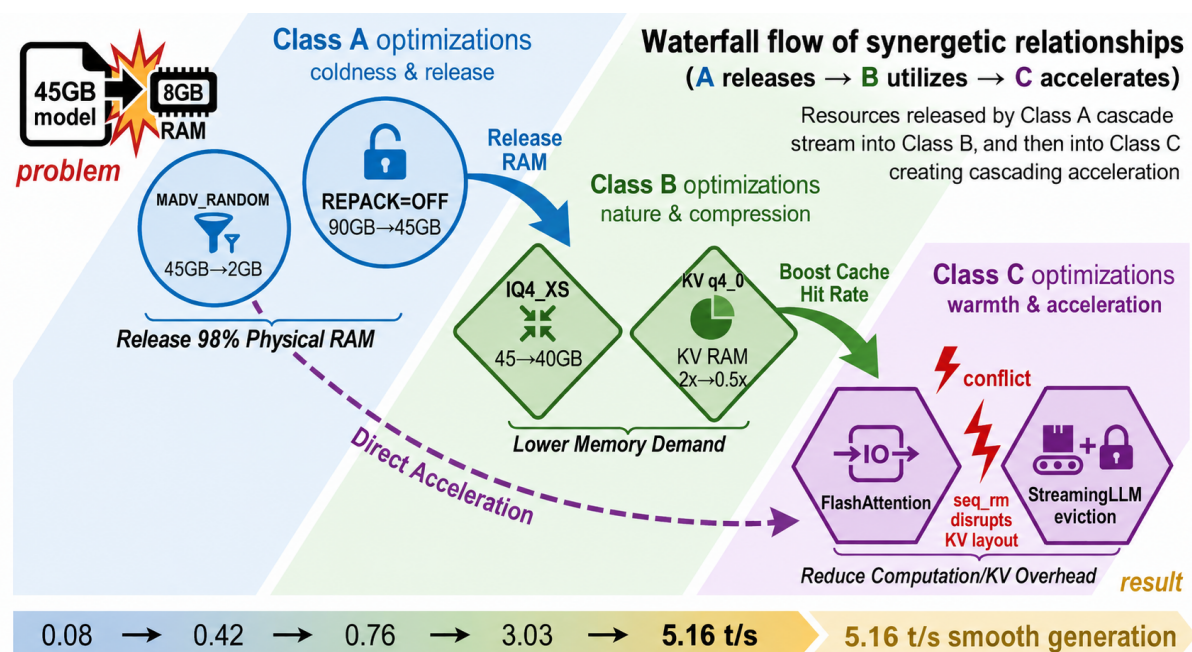


图 6: 优化类别及其相互作用示意。底部吞吐序列来自初赛不同合同下的历史探索，仅用于说明研发路径，不能解释为同一受控实验的累积加速；正式结果见第5节。

组合策略必须重新测量。Mac 正式矩阵中，reclaim 和 residency 单项均未越过拒绝门槛，但 combined cold wall time 回退 19.95%。这个结果并不否定各机制的安全性，而是说明回收产生的 fault 与驻留产生的 admission 工作可能在同一阶段相互放大。

3.8 端侧 Agent Harness 扩展

端侧 Agent 会反复执行“模型决策、工具调用、观测回填、继续生成”。若每一步重新加载权重，或把完整终端输出持续加入上下文，运行时节省的内存和 I/O 会被上层编排重新消耗。SLIM-ARC 因此提供一个最小 Harness，使多轮工具负载遵守同样的资源约束。该扩展不修改计算图，也不属于本文主要性能贡献。

Harness 复用常驻 OpenAI-compatible endpoint，只接受结构化 Shell 工具调用或最终回答。稳定 system prefix 始终保留，历史消息从最近轮次向前装入字符预算，单次工具观测单独截断。若上一轮输出速度低于阈值，下一轮 max_tokens 逐步收缩；命令白名单、工作目录、超时、输出长度、最大轮数和总墙钟共同限制工具闭环。模型事件和工具事件分别记录，从而区分推理慢与外部工具慢。

表 3: 端侧 Agent Harness 的资源策略

端侧问题	设计策略	默认边界
重复加载权重	复用常驻 HTTP 模型服务	CLI fallback 不用于性能结论
工具轮次扩大上下文	稳定 prefix、最近消息窗口、观测截断	8192/2048 字符
Decode 较慢	依据观测 TPS 收缩下一轮输出预算	192 tokens，最低 48
工具阻塞或输出爆炸	白名单、工作目录、超时和输出上限	8s、4 KiB
多轮失控	step 与总墙钟双上限	4 steps、300s

4 系统实现

4.1 与 llama.cpp 的增量集成

SLIM-ARC 基于 llama.cpp 构建, 决赛正式 Mac 镜像固定 upstream revision 360e134。项目没有维护一份长期分叉的完整 llama.cpp 源码, 而是将独立运行时模块存放在 patches/llama-upstream/, 由 scripts/apply-slim-arc.py 复制源文件并对指定 anchor 执行幂等变换。补丁重复应用后生成树的哈希必须保持不变, anchor 缺失时脚本直接失败, 避免上游变化被静默忽略。

集成点分为模型构造和图执行两类。模型构造完成 mmap mapping 与 tensor 迁移后建立 model-owned runtime, 注册普通权重、专家张量和 shared expert 元数据; 图执行在每轮计算前取得 runtime 租约, 调用统一调度器和预取调度器, 在 Router 结果可用后结算 generation。base-line 和 patched 分别构建到独立前缀, 运行脚本为每个变体设置对应 LD_LIBRARY_PATH, 并用动态链接检查确认 llama-bench 解析到自己的 libllama.so。

表 4: SLIM-ARC 主要实现模块

模块	主要对象	职责
slim-arc-runtime	runtime owner、lease	模型所有权、注册、租约和析构屏障
slim-arc-prefetch	prefetch scheduler	阶段预取、generation、专家观测、热度与指标
slim-arc-unified-scheduler	unified scheduler	阶段预算、压力采样、权重/专家额度下发
slim-arc-cgroup-memory	pressure snapshot	严格读取 cgroup v2 current/max
slim-arc-pressure-budget	budget planner	计算 headroom、reserve 和有效额度
slim-arc-page-range	page range	地址溢出检查和内部页对齐
slim-arc-expert-reclaim	reclaim planner	错误 ID、专家切片和 DONTNEED 计划
slim-arc-expert-residency	residency policy	压力滞回、浪费 EWMA、确定性准入
slim-arc-expert-transition	transition model	在线跨层转移计数和候选排序
slim-arc-kv-eviction	KV manager	attention sink、sliding window 与卸载入口

图7把表4中的职责落实到具体源文件和调用关系。它刻意区分三类边：模型拥有的生命周期边、图执行期间的调用边，以及 patcher/实验控制面向上的构建与观测边。

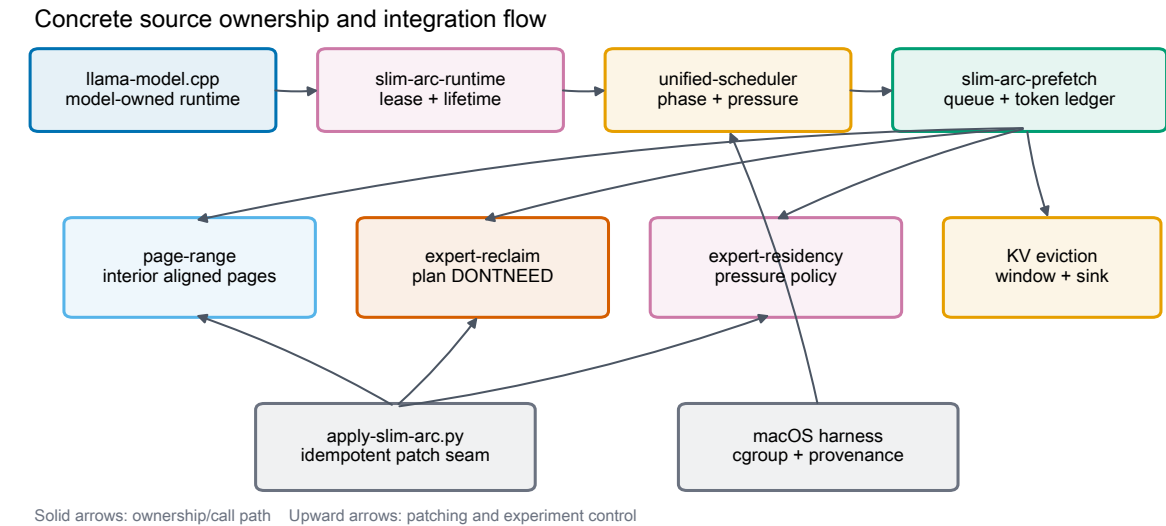


图 7: SLIM-ARC 的源码模块、所有权与集成调用图。上方是模型加载和图执行主路径，中间是独立测试的页范围、回收、驻留和 KV 模块，下方是幂等补丁与实验控制面。

4.2 模型所有权与地址生命周期

早期实现使用进程全局调度器和 mmap registry。该结构无法证明模型销毁后后台 worker 不再访问映射，也使多模型进程的指标和地址相互污染。当前实现把 runtime 作为 llama_model::impl 的末级成员。模型映射和张量注册完成后 runtime 才激活；成员顺序保证 runtime 在 mappings 之前析构。

图执行路径不保存裸调度器指针，而是获取 move-only lease。runtime 销毁时依次执行：从可获取 registry 中移除、拒绝新 lease、等待 in-flight lease 归零、通知 worker 停止、join 全部线程、输出该模型的最终指标、清除张量元数据，最后返回模型析构流程。该顺序将“地址仍有效”和“后台任务仍可能运行”绑定在同一个 owner 内。

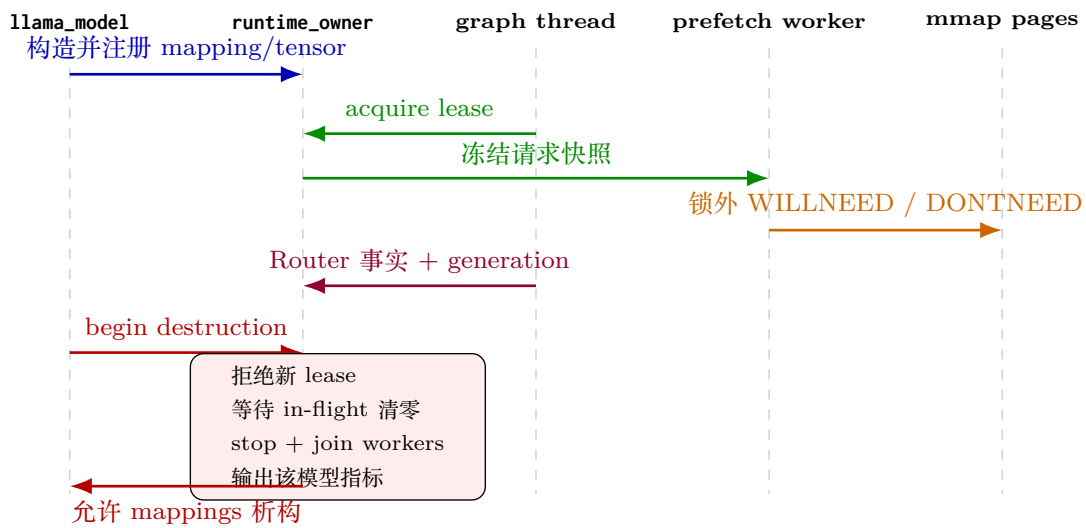


图 8: 模型所有权与并发访问的生命周期时序。运行时在 mmap 之后构造、在 mmap 之前析构；所有页面建议都发生在有效 lease 和映射生命周期内。

4.3 并发调度与一次性结算

预取 worker 使用有界请求队列。每个请求包含 generation、层号、合并后的页范围和冻结预算。producer 在锁内发布或替换尚未领取的请求，worker 在锁内认领并从队列删除，随后释放锁执行 posix_madvise。相同 generation 不会被两个 worker 重复领取；队列满时根据设备模式丢弃最旧未领取请求或拒绝新请求，并增加 dropped/stale 指标。

专家状态由独立互斥锁保护。Router ID、专家张量、上一轮候选、热度和 pending generation 都在锁内复制为值快照；candidate selection、page-range planning 和系统调用均在锁外执行。advice 完成后再以 generation 校验发布成功字节。计算成功时 token-aware settle 会删除恰好一条 pending 记录；失败或无 Router 节点时 unconditional cancel 清理剩余 token，避免 64 个失败轮次耗尽 pending 上限。

算法2给出这条实现路径的关键顺序。入口位于 slim-arc-prefetch.cpp:issue_expert_willneed 负责发出建议，cache_router_experts 负责事实结算，cancel_expert_prefetch 处理未完成轮次。纯规划逻辑分别下沉到 slim-arc-page-range.cpp、slim-arc-expert-reclaim.cpp 与 slim-arc-expert-residency.cpp。因此系统调用 seam 可以在单元测试中注入，策略测试不依赖真实 page cache 状态。

Algorithm 2 图执行期间的预取、结算与回收实现

```

1:  $L \leftarrow \text{runtime.acquire\_lease}()$ 
2:  $q \leftarrow \text{snapshot}(\text{layer}, \text{candidates}, \text{budget}, \text{pressure})$  {锁内复制值}
3:  $g \leftarrow \text{reserve\_generation}(q)$ 
4:  $\text{issued} \leftarrow \text{advise\_outside\_lock}(q, \text{WILLNEED})$ 
5: if 原生 Router 成功且返回有效专家集合  $S$  then
6:    $(\text{hit}, \text{waste}) \leftarrow \text{settle\_once}(g, S, \text{issued})$ 
7:    $\text{plan} \leftarrow \text{build\_reclaim\_plan}(\text{issued} \setminus S)$ 
8:    $\text{advise\_outside\_lock}(\text{plan}, \text{DONTNEED})$ 
9:    $\text{update\_residency}(S, \text{pressure}, \text{waste})$ 
10: else
11:    $\text{cancel\_once}(g)$ 
12: end if

```

4.4 页范围、预算与错误处理

页范围模块使用运行时 `sysconf(_SC_PAGESIZE)`，验证页大小为正且是 2 的幂；起始地址加长度、专家偏移和累计字节均使用 checked/saturating arithmetic。普通权重 WILLNEED 需要覆盖相交页，因此向外对齐；错误专家 DONTNEED 只能回收独占页，因此向内对齐。这两个方向不能混用。

统一预算从严格十进制环境变量读取。SLIM_ARC_TOTAL_BUDGET_MB 的合法范围为 16–1,048,576 MiB，非法值发出警告并恢复历史默认。cgroup 文件采用固定长度读取和 from_chars 解析，拒绝空值、尾随字符、负数、溢出和不一致 current/max。预算不足时记录 skipped bytes；advice 失败记录 errno 相关计数，但不使推理失败。

4.5 运行时配置接口

所有实验优化均通过严格 opt-in 开关实现，未设置时保持兼容路径。表5按功能列出主要配置；完整组合由每次 run manifest 保存。

表 5: 主要运行时配置

功能	环境变量
总体与阶段页策略	SLIM_ARC_DISABLE、DYNAMIC_MADV、DECODE_MADV、SLOW_STORAGE
预取路径	NO_PREFETCH、NO_WEIGHT_PREFETCH、NO_EXPERT_PREFETCH
专家候选	EXPERT_CONF、EXPERT_POP、INLINE_ROUTER、CROSS_LAYER_TOPK
专家常驻与替换	SHARED_MLOCK、EXPERT_HOT_MB、EXPERT_HOT_LRU、EXPERT_HOT_LFRU
回收与压力	EXPERT_RECLAIM_WASTE、EXPERT_RESIDENCY、PRESSURE_ADMISSION
预算与线程	TOTAL_BUDGET_MB、EXPERT_PIPELINE_MB、DECODE_THREADS

4.6 跨硬件实现与适配过程

x86/WSL：功能基线与受限环境。x86/WSL 阶段建立 cgroup v2 隔离、模型量化、mmap、关闭 repack、KV q4_0、FlashAttention 和初始预取路径。三档环境用于验证模型远大于物理内存时的启动和吞吐。该阶段也暴露了并行实验、缓存状态和工作负载不一致造成的数据波动，后续实验改为串行执行并保留原始日志。

RK3588: 从静态建议到阶段策略。 RK3588 为 $4 \times A76 + 4 \times A55$ 的 aarch64 SoC。初次 80B 上机时, 模型通过 SSD 和 mmap 在 8GB 物理内存上运行, 但静态 RANDOM 明显慢于禁用路径。实现随后增加阶段控制器和可配置 Decode MADV: Prefill 改为 SEQUENTIAL 后恢复顺序读取, Decode 在 SEQUENTIAL/NORMAL 之间根据实验选择。后续通过用户级 systemd scope 创建 3–5 GiB cgroup, 并用 memory.current 轮询弥补 5.10 内核缺少 memory.peak 的限制。

Mac/Colima: 可复现的低内存控制面。 Mac 实验运行在 ARM64 Colima VM 内的 Linux 容器, VM 配置为 8 vCPU/16 GiB, benchmark 容器单独限制 1–4 GiB 和 CPU 数, memory.swap.max=0。容器 runner 在执行前后保存 cgroup、GNU time、llama-bench JSON 和 runtime metrics。baseline-first 动态库问题修复后, 构建流程把源码 commit、build context hash、patched tree hash、模型 SHA 和 image ID 写入 manifest, 避免同名 tag 或错误链接污染 A/B。

Raspberry Pi 5: 慢存储设备策略。 Raspberry Pi 5 使用 4 核 A76、4 GiB 内存以及 USB 存储上的 NTFS-3G/FUSE。该链路的主要问题不是带宽上限, 而是同步缺页经 FUSE 和 USB bridge 往返的固定延迟。系统为此实现 shared expert mlock、resident-only hot cache、16 MiB 总预算、weight/expert prefetch 独立开关、跨 token LRU 和文件级 expert advice。实验否决了 LFRU、二次命中准入、NTFS3 mmap 和提前文件预读, 最终保留“确定复用页常驻、投机 I/O 关闭或严格限额”的策略。

华为 HiDevLab: aarch64 可移植路径。 HiDevLab 独立目录中的环境可以枚举 Ascend 910B4, 但缺少完整 CANN/torch_npu 用户态栈。本项目没有修改系统驱动, 也没有占用其他比赛的 NPU 进程。为获得与系统相关的数据, 项目在该环境编译 llama.cpp aarch64 CPU-only 路径, 并以 OLMoE Q2_K 执行 baseline、patched-default 和多种页策略 A/B。该实现证明 POSIX 路径可移植, 不等同于 NPU kernel 适配。

4.7 端侧 Agent Harness 实现

Harness 位于 scripts/agent/edge_agent.py。正式路径通过 OpenAI-compatible HTTP endpoint 发送消息并解析 SSE, 模型只允许返回 {"tool": "shell", "args": [...]} 或 {"final": "..."}。工具参数以 argv 数组传入 subprocess, 不启用 shell=True; 程序名、相对路径、工作目录和符号链接目标均在执行前验证。

上下文构造先固定 system prefix, 再从最近消息向前装入字符预算; 工具输出采用头尾保留的居中截断。输出预算以最近一次估算 TPS 更新, 低于 2.0 token/s 时乘 0.65, 最低 48 tokens。每轮生成事件记录上下文字符、最大 token、请求耗时、TTFT 和估算 TPS; 工具事件记录 argv、返回码、耗时、输出长度和截断标志。CLI fallback 便于功能诊断, 但会重复加载模型, 不用于性能比较。

4.8 指标、测试与复现实现

patched 进程结束时输出一条带 schema 版本的 [SLIM-ARC-RUNTIME] 行。字段包括普通权重请求/发出/跳过字节、expert issue/hit/waste、pending 与 stale 请求、页范围合并、回收候选/调用/跳过/失败、驻留样本/准入/跳过/fallback 和压力状态。解析器拒绝重复行、缺失字段、未知字段、非十进制或溢出; baseline 必须不输出该行。

C++ 单元测试覆盖 page range、pressure budget、prefetch budget、expert reclaim、expert residency、unified pressure 和 runtime lifecycle, 并可启用 ASan/UBSan。Python 测试覆盖 patcher 幂等性、Mac controller、runtime metrics、evidence aggregation 和 Agent Harness。实验运行额外保存模型哈希、源码身份、镜像身份、资源限制、workload 和原始输出, 使表格中的每一行都能回到对应目录。

5 实验评估与结果分析

5.1 实验设置

5.1.1 研究问题

实验围绕五个研究问题展开。RQ1 检验远大于物理内存的 MoE 模型能否在 no-swap 条件下完成推理；RQ2 检验阶段感知页建议能否消除静态策略的负优化；RQ3 检验专家预测、驻留、回收和预算的收益来自哪些模块；RQ4 分析存储、内存压力和缓存状态如何改变策略选择；RQ5 验证运行时能否迁移到不同 aarch64 平台，并为端侧 Agent 提供功能闭环。

5.1.2 硬件与软件环境

表6列出五类实验平台。不同平台的 CPU、文件系统、缓存控制和资源限制不同，实验只在同设备、同模型、同 workload 和相同缓存状态内计算提升率。

表 6: 实验平台

平台	CPU/内存	存储与隔离	主要实验目的
WSL/x86 RK3588	i9-13900H, 8–32 GiB cgroup 4×A76+4×A55, 7.8 GB	NVMe, cgroup v2 SSD 约 2.1 GB/s; 2.5–5 GiB user scope	初赛基线、量化、KV/FA 与历史消融 动态 MADV、3 GiB 主实验、长上下文和专 家预测
Mac/Colima Raspberry Pi 5	ARM64 VM 8 vCPU/16 GiB 4×A76, 4 GiB	容器限 1–4 GiB, swap 0 USB NTFS-3G/FUSE, 运行期 no- swap	2 GiB 正式矩阵、机制筛选和终态复核 慢存储驻留、预算、LRU 和文件系统实验
HiDevLab	aarch64; 910B4 可枚举	独立目录, llama.cpp CPU-only	POSIX 路径移植和小 MoE A/B

5.1.3 模型、工作负载与指标

主模型为 Qwen3-Next-80B-A3B-Instruct Q4_K_M，文件大小 48,410,988,384 B，决赛固定 SHA-256 为 d103b2733ec1012a52d01edda66b7e5c24ae50508c9f99f5297ea459ef3c061a。Qwen3-4B 用于 Dense 基线、KV 与 FlashAttention；OLMoE-1B-7B 用于小 MoE Router 链路和 HiDevLab A/B。初赛 IQ4_XS 结果仅用于量化边界。

llama-bench 的 Prefill (pp) 和 Decode (tg) 均以 token/s 表示；wall time 表示完整 benchmark 时间。内存同时报告 cgroup memory.peak/current 与进程 RSS，两者不互换。I/O 指标包括 major faults、file-system input、weight/expert issued bytes、skipped bytes 和 in-flight peak；专家指标包括 samples、hit、waste、reclaim、residency 和 pressure 状态。cold、warm、mixed cache 分开分析。

正式 Mac 矩阵固定 80B Q4_K_M、2 GiB、4 CPU、swap 0、pp64/tg16，每个配置执行两轮 cold/warm，共 20 次。RK3588 的 3 GiB 主实验固定 80B、4 threads、pp128/tg64、KV q4_0、MemorySwapMax 0。Pi 的 80B 主序列固定 4 threads、pp16/tg16、USB/FUSE、运行期 no-swap。其他工作负载在对应表中单独标注。

5.2 主实验结果

5.2.1 x86/WSL：初赛核心性能与优化链

初赛首先在 Intel i9-13900H、32 GB DDR5、NVMe SSD 和 Ubuntu 22.04/WSL2 环境中建立 80B 推理基线，并使用 cgroup v2 构造 8、16 和 32 GiB 三档内存。该平台承担两项核心验证：其一，检验 SLIM-ARC 能否在模型文件显著大于内存上限时改善 Decode；其二，分析权重量化、缓存

容量与 FlashAttention 如何改变不同运行区间的瓶颈。决赛实验并非替代这些结果，而是在 Mac、RK3588 和 Raspberry Pi 5 上进一步检验其跨设备边界。

表7给出 8 GiB 下两组同合同配对结果。pp4/tg1 合同中，SLIM-ARC 将 Prefill 从 0.22 提高到 0.25 token/s，将 Decode 从 0.08 提高到 0.43 token/s，Decode 提升 437.5%。在较长的 pp16/tg4 合同中，Decode 从 0.08 提高到 0.29 token/s，提升 262.5%；与此同时，Prefill 从 0.63 降至 0.28 token/s。后者说明稀疏 Decode 的收益不能外推到密集 Prefill，阶段识别与分阶段策略是必要设计，而不是实现细节。

表 7: WSL 8 GiB 下的同合同 80B 配对结果

合同	配置	Prefill	Decode	Decode 提升
pp4/tg1	baseline	0.22	0.08	reference
pp4/tg1	SLIM-ARC	0.25	0.43	+437.5%
pp16/tg4	baseline	0.63	0.08	reference
pp16/tg4	SLIM-ARC	0.28	0.29	+262.5%

内存容量与量化格式决定了可被页缓存复用的权重比例。表8中，IQ4_XS 在 16 GiB 下达到 Decode 2.27 ± 0.38 token/s，在 32 GiB 下达到 3.03 ± 0.29 token/s；同一 32 GiB 档的 Q4_K_M 为 2.68 ± 0.23 token/s。IQ4_XS 将模型文件由约 45 GB 收缩到约 40 GB，使 32 GiB 环境能够缓存更多权重页。三档结果用于刻画容量和量化边界，不与 8 GiB 配对实验拼接成单一倍率。

表 8: WSL 内存容量、量化格式与 80B 吞吐

内存	模型格式	线程	Prefill	Decode
16 GiB	IQ4_XS	8	—	2.27 ± 0.38
32 GiB	IQ4_XS	8	4.44 ± 1.53	3.03 ± 0.29
32 GiB	Q4_K_M	8	—	2.68 ± 0.23

在固定 32 GiB、8 threads、KV q4_0 和 warm cache 的 Attention 消融中，未启用自动选择时 Decode 为 3.01 token/s，-fa on 达到 3.90，-fa auto 达到 5.16 token/s，相对 3.01 提升 71.4%。该实验表明，当权重换页压力下降后，Attention I/O 会成为新的主要开销，计算融合能够继续提高热缓存区间的吞吐。

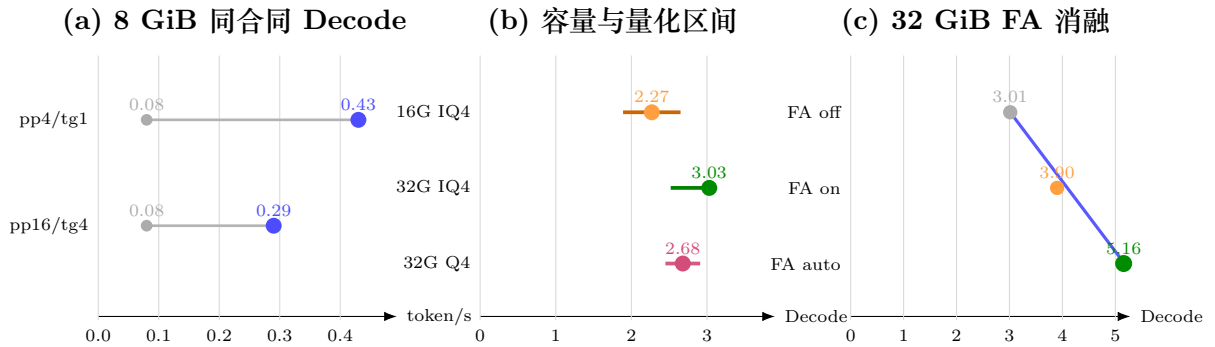


图 9: WSL 初赛主实验的三个证据层次。左：同合同 baseline-SLIM-ARC 配对；中：内存容量与量化格式的运行区间，横线为均值加减标准差；右：固定热缓存合同下的 FlashAttention 消融。三组数据不跨合同合并计算倍率。

WSL 主实验形成了完整的因果链：8 GiB 配对结果验证页管理在强内存约束下能够显著改善 Decode；16–32 GiB 结果表明缓存容量和模型文件大小共同决定稳定吞吐；固定 32 GiB 的 Attention 消融则说明瓶颈会从权重 I/O 转移到算子 I/O。这一观察直接推动了决赛阶段的设备化策略：Mac 用于验证更小物理内存边界，RK3588 用于验证阶段页建议，Raspberry Pi 5 用于验证慢存储下的驻留与预算。

5.2.2 48.41 GB 模型在 2 GiB no-swap 环境中运行

Mac/Colima 的 8 月 12 日正式矩阵 20/20 成功，所有配置的 cgroup peak 均达到 2 GiB，但没有 OOM 和 swap。8 月 17 日终态复核在 cold cache、4 CPU、pp64/tg16 和 A24 配置下成功退出，Prefill 为 3.908455 token/s，Decode 为 0.709095 token/s，wall time 58.06 s，major faults 402,118。运行时记录 816 个专家样本，expert advice/issued/waste 均为 0，证明禁用投机专家预取的配置在最终镜像中生效。

该实验直接回答 RQ1：mmap 与 page-cache 逐出使模型文件可以远大于 cgroup 上限，no-swap 并不要求全部权重常驻。其代价是大量同步缺页与存储读取，因此 2 GiB 结果表示可运行性和确定资源边界，不表示模型文件被压缩为 2 GiB。

5.2.3 RK3588 3 GiB cgroup 主实验

表9给出队友完成的 F1–F5 实验。3 GiB 下 baseline Decode 只有 0.70 token/s；SLIM-ARC 达到 2.21 token/s，提升 216% (3.16 倍)，Prefill 从 3.85 提高到 4.40 token/s。增加 KV eviction 后仍达到 4.19/2.12 token/s，说明短工作负载的主要收益来自权重页策略，而非 KV 驱逐。

表 9: RK3588 受限内存主实验 (80B, 4 threads, pp128/tg64, swap 0)

MemMax	配置	Prefill	Decode	相对同档 baseline
3 GiB	baseline	3.85	0.70	reference
3 GiB	SLIM-ARC	4.40	2.21	pp +14%, tg +216%
3 GiB	SLIM-ARC + KV eviction	4.19	2.12	pp +9%, tg +203%
2.5 GiB	baseline	3.97	1.91	reference
2.5 GiB	SLIM-ARC + KV eviction	4.26	2.21	pp +7%, tg +16%

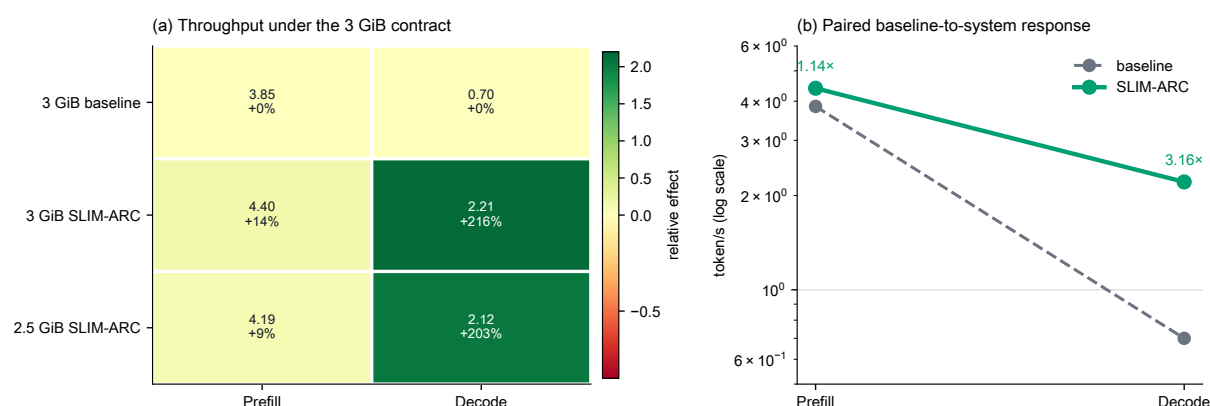


图 10: RK3588 3 GiB cgroup 的归一化性能温度图与配对响应图。左图同时编码绝对吞吐和同档增益，右图在对数轴上显示 baseline 到 SLIM-ARC 的 Prefill/Decode 迁移。

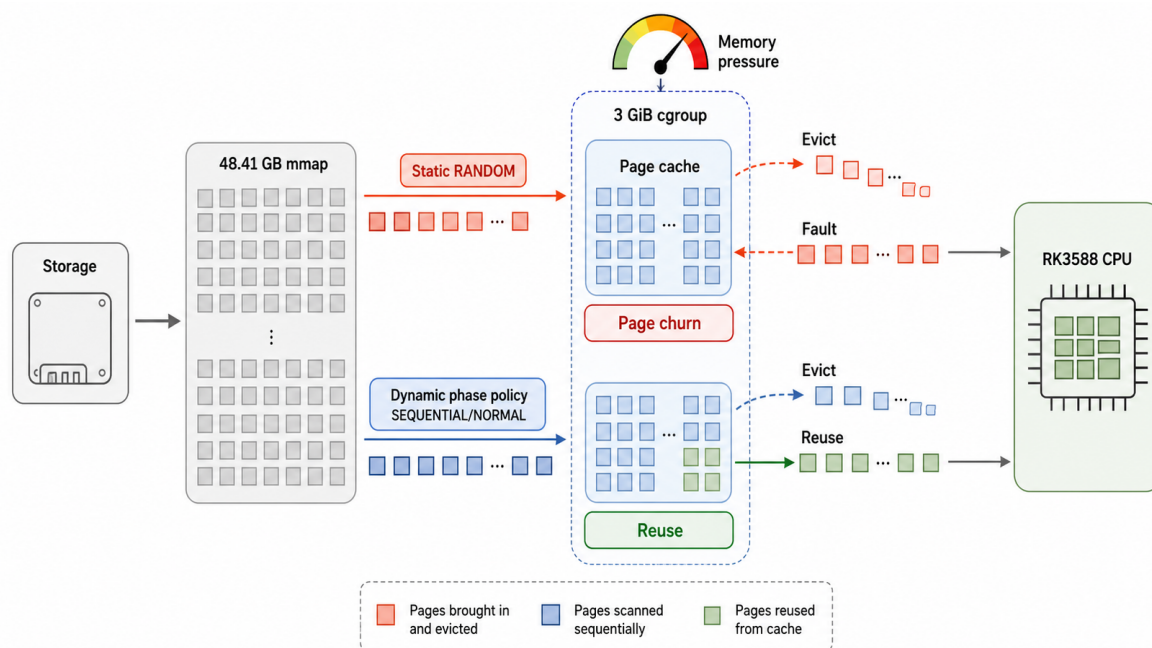


图 11: RK3588 3 GiB 强压力下的页访问机制。静态 RANDOM 将模型扫描拆为页抖动与同步缺页；阶段策略在 Prefill 使用有序扫描，并在 Decode 维持可复用页。图中不承载实验数值，数值以图10为准。

2.5 GiB baseline 的单次 Decode 反而高于 3 GiB baseline，说明 page-cache 初始状态和驻留窗口会放大单次实验波动；该行只在 2.5 GiB 档内比较，不能据此推导“内存越小越快”。五组均未触发 OOM，内存压力表现为页缓存逐出和性能变化。

5.2.4 RK3588 动态页建议恢复

静态 RANDOM 的短工作负载只有 0.44/0.26 token/s，禁用 SLIM-ARC 为 2.74/1.41。Prefill 改为 SEQUENTIAL 后提高到 2.82 token/s，但 Decode 保持 RANDOM 时仍只有 0.25；Decode 改为 SEQUENTIAL 后达到 2.81/1.35，最终动态全 SEQUENTIAL 为 2.84/1.40，与禁用路径差距缩小到 0-4%。这条消融链直接证明阶段识别而非“SLIM-ARC 开或关”决定了结果。

表 10: RK3588 动态 MADV 消融 (80B, p32/n16/t4)

策略	Prefill	Decode	解释
静态 RANDOM	0.44	0.26	顺序访问被拆成同步小读
禁用 SLIM-ARC	2.74	1.41	upstream reference
Prefill SEQUENTIAL / Decode RANDOM	2.82	0.25	只恢复 Prefill
Decode SEQUENTIAL	2.81	1.35	恢复 Decode
Decode NORMAL	2.57	1.41	Decode 快，Prefill 较低
动态全 SEQUENTIAL	2.84	1.40	保留运行时且追平 reference

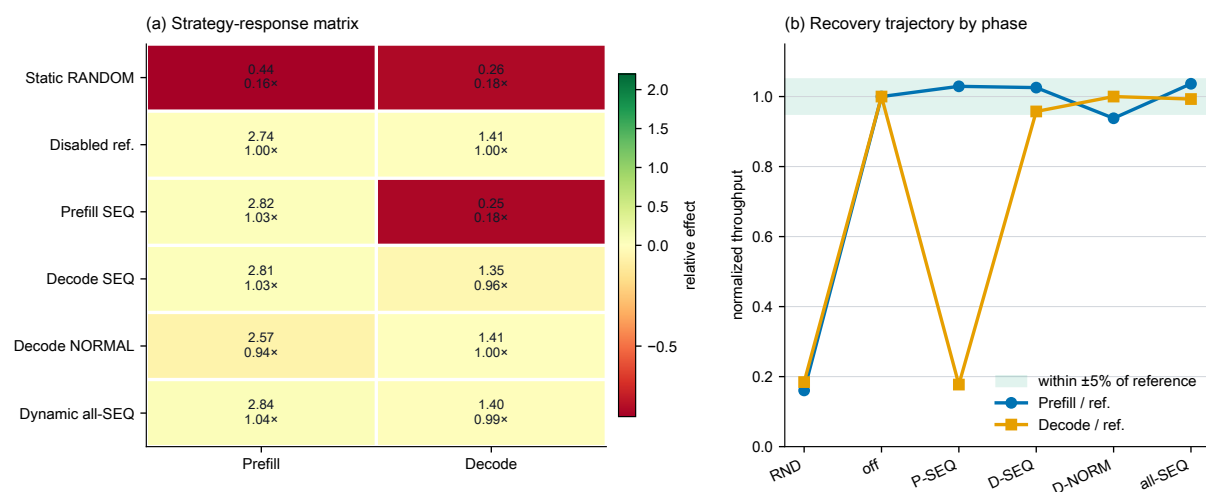


图 12: RK3588 动态页建议的策略响应温度图和恢复轨迹。单独修复 Prefill 不能恢复 Decode；将阶段策略扩展到 Decode 后，两项吞吐才共同进入 reference 的 $\pm 5\%$ 区间。

5.2.5 Raspberry Pi 5 慢存储主实验

Pi 的核心策略从“扩大预取”转为“保留确定复用页并停止无收益投机”。A24 关闭 expert prefetch 后为 0.201104/0.055614 token/s、547 s。A28 加入 shared expert 与 resident hot512 后达到 0.204023/0.063486、511 s，即 Decode +14.16%、wall -6.58%。A29 将总预算限制为 16 MiB，issued/in-flight weight bytes 均减少 98.44%，吞吐近似不变。A32 完全关闭 weight prefetch 后 wall 505 s；A34 的 512 MiB 跨 token LRU 达到 Decode 0.065856、wall 503 s。

表 11: Raspberry Pi 5 的慢存储策略演进

编号	策略	Prefill	Decode	Wall(s)
A24	no expert prefetch	0.201104	0.055614	547.0
A26	resident hot512	0.200578	0.060726	522.5
A28	shared + hot512	0.204023	0.063486	511.0
A29	A28 + 16 MiB budget	0.204283	0.063585	510.5
A32	no weight prefetch	0.203642	0.065246	505.0
A34	512 MiB cross-token LRU	0.203585	0.065856	503.0

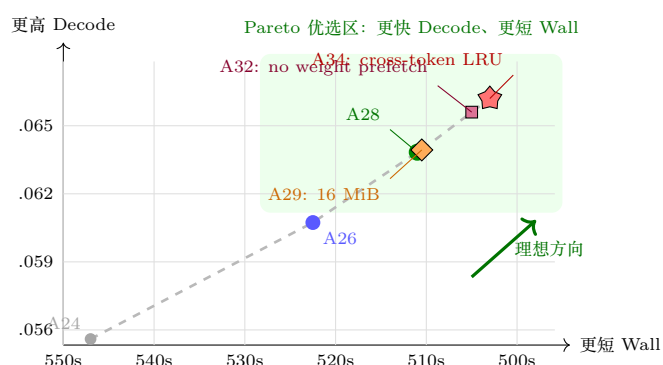


图 13: Raspberry Pi 5 的 Decode-Wall 多目标 Pareto 图。预算收缩、关闭无收益预取和跨 token LRU 依次把运行点推向右上角。

主实验共同回答 RQ4: RK3588 3 GiB 的强压力需要有界预取避免同步抖动; Pi 的 FUSE/USB 链路则更适合零投机和常驻复用页。两者都使用 mmap 和动态机制, 但最终策略方向不同。

5.3 消融实验

5.3.1 Mac 正式机制矩阵

Mac 五配置两轮 cold/warm 的结构化结果由 hash 绑定的 finals-results.json 导入。所有配置的 memory.peak 都为 2 GiB, 因此本轮不能证明峰值内存降低。cold 条件下 reclaim 相对 patched-control 的 wall 改善 4.39%、Decode 提高 6.31%, 但回收计数为 0, 变化不能归因于 DONTNEED; residency 仅改善 0.21% wall; combined wall 回退 19.95% 并被拒绝。warm 条件下 baseline 快于 patched-control, 进一步说明缓存状态足以改变排序。

表 12: 固定 Mac 2 GiB/4 vCPU/no-swap 协议下的两轮聚合结果 (由 finals-results.json 派生)。

配置	缓存	峰值内存 (GiB)	Wall (s)	Decode (t/s)	专家浪费 (B)
baseline	cold	2.000	69.530	0.600	0
baseline	warm	2.000	53.535	1.232	0
patched-control	cold	2.000	66.300	0.689	0
patched-control	warm	2.000	60.300	0.821	0
patched-reclaim	cold	2.000	63.390	0.732	0
patched-reclaim	warm	2.000	58.080	0.844	0
patched-residency	cold	2.000	66.160	0.692	0
patched-residency	warm	2.000	63.190	0.712	0
patched-combined	cold	2.000	79.525	0.668	0
patched-combined	warm	2.000	63.265	0.760	0

表 13: 由两轮 cache-separated promotion gate 聚合的功能分类 (由 finals-results.json 派生)。

机制	cold	warm	总体
错误专家回收	kept_opt_in	kept_opt_in	kept_opt_in
专家驻留	kept_opt_in	kept_opt_in	kept_opt_in
组合策略	rejected	kept_opt_in	rejected

finals-results.json SHA-256: 843b54ada66bfea0d52ec55efcbf8f91b592d588cbff0204efc40400196c655

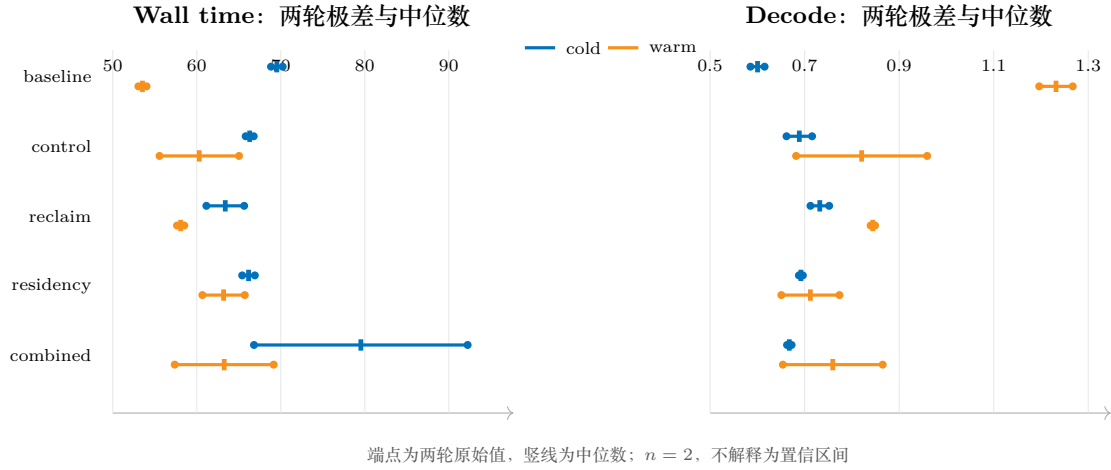


图 14: Mac 2 GiB 正式矩阵的两轮区间图。cold 与 warm 分开显示, 组合策略的 wall 波动和 warm 配置的离散性清晰可见。

5.3.2 专家预测、门控与预算

RK3588 的 temporal predictor 准确率为 34.9%, 明显高于约 2% 的随机参考和 4.5% 的 spatial predictor。confidence gating 将 expert issued 从 27,836.7 MB 降至 12,123.6 MB, hit rate 从 31.05% 提高到 55.35%, 速度保持约 1.9 token/s; confidence 与 budget 组合达到 54.00% hit 和 2.1 token/s。相反, popular-16 将 issued 增加到 72,634.4 MB、hit 降至 19.31%, 因此被拒绝。

Mac A18 的 inline Router + hot512 相对对照带来 Decode +10.35%、wall -7.77%; A20 layer-local pipeline 将 fault 降低 4.09%、I/O 降低 5.10%, 但 Prefill 回退 2.10%; A22 cross-layer Router Top2 的命中率达到 88.56%, Decode +9.96%、wall -7.77%, 仍未超过更低开销候选。A24 关闭 expert speculative prefetch 的三轮中位数为 Prefill +10.66%、Decode +22.45%、wall -15.72%, 说明在该合同下额外专家 I/O 的边际收益不足。

5.3.3 线程、readahead 与普通权重预取

Mac A4 将 Prefill/Decode 线程设为 8/6, Decode 提高 20.23%、wall 降低 5.91%; A10 进一步确认 6 个 Decode 线程是 wall 与吞吐的折中点。A11 的 128/192/256 KiB readahead 形成 Pareto 前沿, A16 的 sustained 256 KiB 配置使 Prefill 提高 13.39%、Decode 提高 2.10%、wall 降低 6.39%。该参数不被硬编码, 因为 Pi 和 RK3588 的链路不同。

A32/A33 给出明确跨设备反例: Mac 上 weight prefetch on 为 Decode 1.083、wall 92.19s, off 为 1.022/96.14s; Pi 上 on/off wall 均约 505s, 而 off 消除了 60.59 MiB 投机 I/O。系统因此保留设备配置, 而不定义跨平台默认值。

5.3.4 KV、FlashAttention 与量化

RK3588 Qwen3-4B 上 FlashAttention auto 的 Decode 为 6.90 token/s, 关闭后为 3.49, 说明 ARM 小模型的 Attention 路径受益明显; KV q4_0 为 5.05, 短上下文下反量化开销超过节省的带宽。初赛 80B 长上下文实验确认 KV eviction 在 8192-16384 context 下可以稳定执行, 但 4 GiB cgroup 的 4096/8192 Prompt 在 600s 内均未完成 Prefill, 原因是权重换页与大 Prompt 扫描, 而非 OOM。

IQ4_XS 在初赛显示更小模型文件和较高热缓存吞吐, 但 GSM8K 10 题得到 0/10; Q4_K_M + KV q4_0 为 5/10, Qwen3-4B Q4_K_M 为 15/20。样本规模不足以给出正式精度排名, 但足以阻止 IQ4_XS 成为默认方案。

5.3.5 被拒绝的优化

Self-draft speculative decoding 将 80B Decode 从 3.01 降至 1.41 token/s (-53.2%), accept rate 33.3%, 因此不再进入主线。Mac expert RANDOM 将 Decode 从 0.801 降至 0.262、wall 从 55.90 增至 96.42s; expert NORMAL 也回退。Pi 的 LFRU 使 wall 增加 1.50%, 二次命中准入使 Prefill 下降约 54%, NTFS3 虽提高 direct-read 57.28%, 真实 mmap Prefill 却下降 65.70%, exact expert fadvise 为 -0.2517% 。这些结果共同说明, 局部 cache、带宽或命中指标不能替代端到端测量。

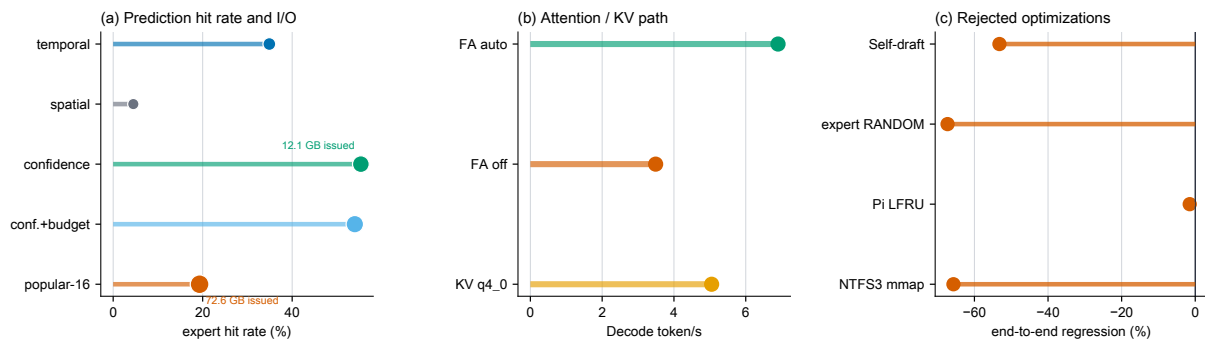


图 15: 消融实验多指标总览。左: 专家预测命中与投机 I/O 的联合代价; 中: FlashAttention 与 KV 量化对 ARM Decode 的影响; 右: 由端到端指标否决的优化。图中只连接同一实验合同内可比较的数据。

5.4 设备专项与补充实验

RK3588 长上下文、压力与内存。 RK3588 的长上下文动态策略在 p512/n128 下达到 6.09/2.08 token/s, 接近禁用路径的 6.33/2.15。3 GiB RSS 补测中, baseline VmHWM 4324 MB, SLIM-ARC 为 3986 MB, SLIM-ARC + KV eviction 为 3951 MB; 三个 user scope 的 memory.current 都触及 3072 MB。由于文件页可能计费到外部 session cgroup, 进程 RSS 大于 scope current, 两者不能直接相减为“未计费内存”。

4B 五轮对话中, baseline Generation 均值为 4.52 token/s, SLIM-ARC + KV eviction 为 5.20 (+15%), Prompt 基本持平; H1 首轮输入含 shell echo -e artifact, 因此结果列为探索性。4 GiB、5 GiB 和并发压力实验还分别观察到 Decode +13.3%、+15.7% 和 +5.8%, 完整合同见附录。

Raspberry Pi 5 替换策略与质量边界。 A34 512 MiB LRU 相对旧 cache 将 hits 提高 6.53%、evictions 降低 36.31%; 384 MiB 使 evictions 相对 A34 增加 121.10%。A37 LFRU 虽将部分 cache counter 改善, 却使 nonresident scan 增加 56.33% 并降低吞吐。A49 top-8 相对 A50 top-10 在单次 pp4/tg1 中 Prefill +17.56%、Decode +10.53%、wall -3.30% , 但一题算术质量检查无法区分二者, 因而只保留为精度门之前的研究开关。

HiDevLab aarch64 CPU-only 实验。 HiDevLab 环境确认 aarch64、Ascend 910B4 32 GiB HBM 和驱动可枚举, 但系统缺少匹配 CANN toolkit、libascendcl.so 和完整 torch_npu。因此性

能实验使用 OLMoE-1B-7B Q2_K 的 llama.cpp CPU-only 路径, 固定 16 threads、pp64/tg16、3 repetitions 和 warm cache。

baseline 为 100.724597/51.340323 token/s、wall 3.2546 s; patched-default 为 101.175889/45.254024、3.415535 s; A24 为 100.602680/48.184150、3.373611 s。A24 相对 patched-default 的 Decode 提高 6.47%，但相对原生 baseline 仍低 6.15%。该结果证明 POSIX 路径可移植，也说明 2.56 GB 小模型在内存充足和 warm cache 时没有足够的换页压力产生系统收益。

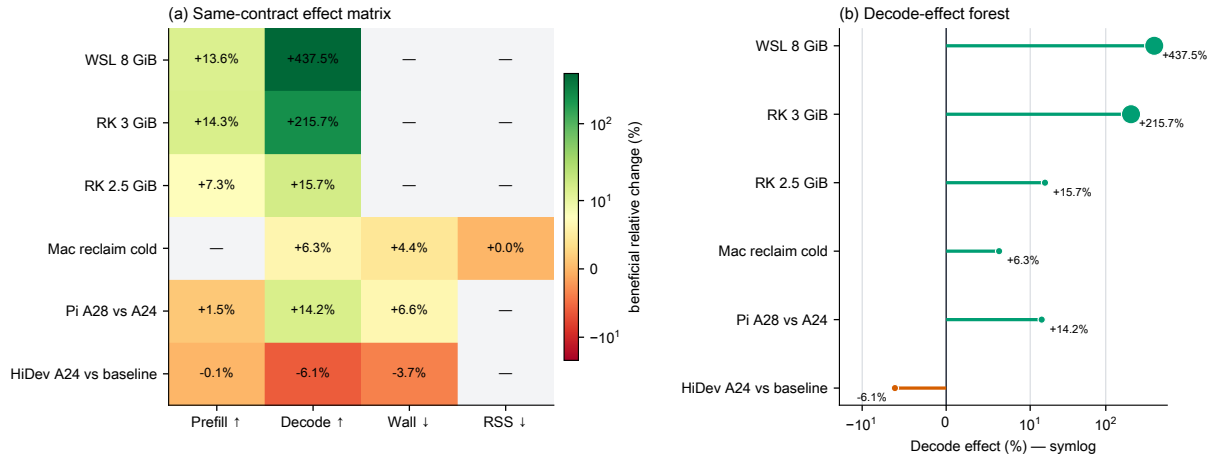


图 16: 跨设备同合同效应矩阵与 Decode 效应森林图。左图在各设备内部合同中编码有利方向变化, 右图使用对称对数轴呈现效应量级; 空白表示该合同没有可比指标, 不作跨设备绝对吞吐比较。

端侧 Agent Harness 功能实验。确定性本地 endpoint 首轮返回 `uname -m` 工具请求, Harness 在受限工作目录执行并回填观测, 第二轮返回最终答案。首轮估算速度 1.999 token/s, 触发 `max_tokens` 从 192 降至 124; 若持续低于阈值, 最低可降至 48。工具返回码为 0, 输出未截断。图17展示实际 JSONL 事件。

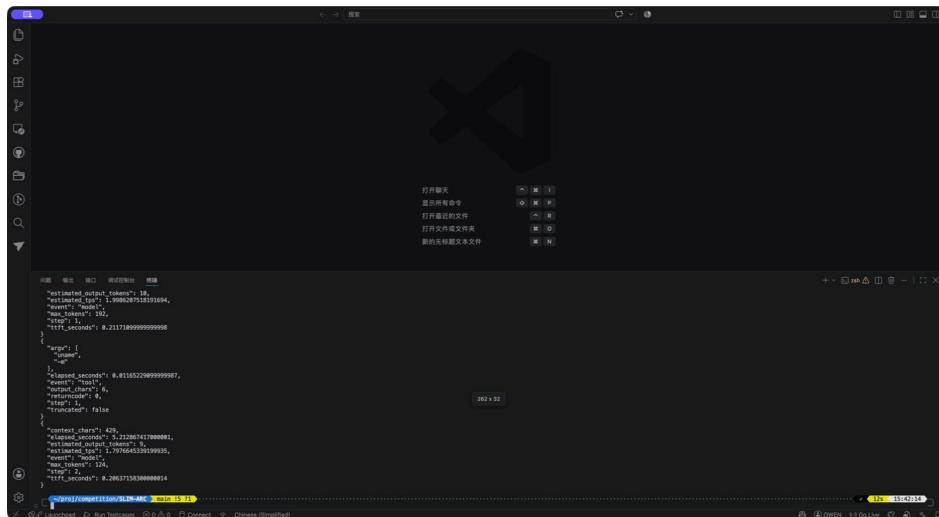


图 17: Agent Harness 的模型、受限 Shell 和观测回填闭环。该图验证功能、边界和指标, 不代表真实 80B TPS 加速。

5.5 综合结果分析

实验支持四项总体结论。第一，mmap、关闭 repack 和页缓存逐出使 48.41 GB 模型能够在 2–8 GiB 环境中运行；“可运行”与“全部权重常驻”是不同目标。第二，页建议的收益由阶段和存储决定：RK3588 的动态 SEQUENTIAL 消除了静态 RANDOM 负优化，Pi 则进一步关闭投机预取。第三，专家管理的关键不是单独追求命中率，而是减少无收益 I/O 并为确定复用页保留有限容量：A22 高命中和 popular-16 负结果都支持这一点。第四，组合机制必须重新验证。Mac combined 的回退、KV q4_0 在 ARM 小模型上的变慢和 LFRU 的负结果说明，局部正确性或单项指标不能推出端到端协同。

SLIM-ARC 最强的正向场景出现在内存压力与同步缺页足够强的设备内合同，例如 RK3588 3 GiB 和 Pi 4 GiB 慢存储；在内存充足、模型较小或 warm cache 的 HiDevLab OLMoE 环境中，运行时管理成本可能高于收益。系统因此将策略保留为设备化配置，并把负面结果作为适用范围的一部分。

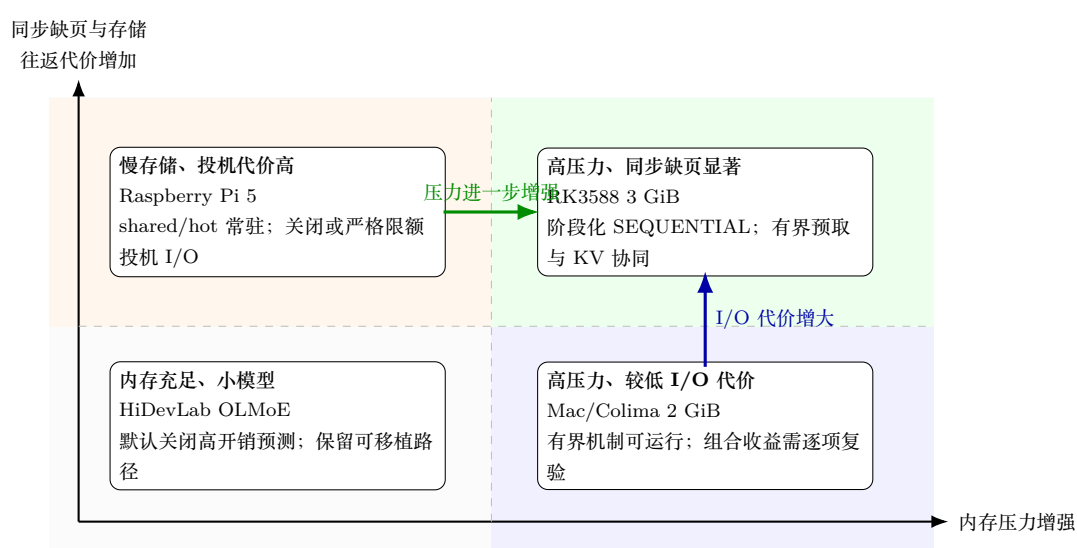


图 18: 由实验归纳的设备策略适用域。该图不进行跨设备吞吐排名，而是把内存压力与同步缺页代价映射到不同策略：高压强调阶段化和有界性，慢存储强调确定复用并抑制投机，内存充足时则避免运行时管理开销。

6 总结、局限性与未来工作

6.1 工作总结

本文提出并实现了面向内存受限端侧系统的 MoE 推理运行时 SLIM-ARC。系统把稀疏 Router 访问转化为一个可观测、可撤回且受预算约束的物理页管理闭环：阶段控制器选择权重页策略，层级与 Router 信号生成候选，统一预算和压力状态决定准入，实际 Router 结果结算命中与浪费，错误专家的完整内部页被安全回收，shared/hot 专家则在设备允许时进入有限驻留集合。mmap、关闭 repack、KV q4_0、StreamingLLM eviction 和 FlashAttention 与该闭环共同构成完整的端侧推理路径。

工程实现将运行时归属于模型对象，并以租约连接图执行和析构过程。有界 generation 队列、锁内快照、锁外页建议、严格算术、异常回退和版本化指标使性能机制具有明确的生命周期和归因边界。幂等 patcher、独立 baseline/patched 构建、模型与镜像身份、cgroup 记录和结构化结果共同支持跨平台复现。端侧 Agent Harness 进一步说明，上层多轮工具负载可以通过常驻服务、短上下文、输出预算和受限 Shell 服从同一资源目标。

6.2 主要实验结论

初赛 WSL 主实验首先证明了系统优化链的有效性：8 GiB 同合同下 Decode 从 0.08 提高到 0.43 token/s，固定 32 GiB warm-cache 合同下 FlashAttention auto 将 Decode 从 3.01 提高到 5.16 token/s。决赛实验随后把验证扩展到更严格和更多样的设备：48.41 GB 模型在 Mac/Colima 的 2 GiB、4 CPU、no-swap 环境完成推理；RK3588 的 3 GiB cgroup 主实验将 Decode 从 0.70 提升至 2.21 token/s，阶段消融又将静态 RANDOM 的 0.44/0.26 恢复到 2.84/1.40 token/s；Raspberry Pi 5 上 shared/hot 专家驻留相对 A24 将 Decode 提高 14.16%、wall 降低 6.58%，16 MiB 预算进一步将投机 weight I/O 降低 98.44%。这些结果分别对应初赛性能基线、极限可运行性、强内存压力、阶段访问差异和慢存储复用五类问题。

负面结果同样确定了系统边界。Mac 正式矩阵中的 combined cold wall 回退 19.95%，HiDevLab OLMoE A24 仍比原生 baseline 慢 6.15%，RK3588/Pi 的 expert RANDOM、popular-16、LFRU、NTFS3 mmap 和提前文件预读均未产生端到端收益。它们说明命中率、缓存计数、direct-read 带宽或机制正确性都不能替代匹配合同的 wall 与吞吐测量。

6.3 局限性

SLIM-ARC 仍存在四方面限制。第一，POSIX 页建议是 best-effort，内核 readahead、cgroup page-cache 计费、FUSE、USB bridge 和设备内部缓存都会改变实际效果；无 root 设备上的 cold cache 只能近似构造。第二，错误专家回收向内对齐完整页面，保守地跳过边界页和不规则布局，安全性优先于覆盖率。第三，pressure、waste 和 popularity 阈值来自确定性工程规则，尚未在更多模型、序列分布和长期 workload 上校准。第四，当前主要路径是 CPU-only llama.cpp；HiDevLab 只验证 aarch64 可移植性，未完成 CANN/NPU kernel、HBM/DDR 分层或 910B 性能评测。

实验范围也限制了结论强度。Mac 正式矩阵只有两轮 cold/warm，所有配置都触及 2 GiB cgroup 上限，因而无法证明峰值下降；部分 RK3588 和 Pi 优化只有单次或两次运行，只用于设备内趋势与机制选型；GSM8K 和 top- k 质量检查样本不足以给出精度保证。跨设备结果不能合并成单一加速比。

6.4 未来工作

后续工作将围绕能够直接减少每 token 读取量的方向展开。第一, 对 routed experts 进行连续打包和更低比特量化, 同时保护 Router、Norm、Attention 与 shared expert, 评估 read bytes/token、吞吐和质量的联合变化。第二, 在真实 routing trace 上比较 per-layer LRU、LRU-K、ARC 和 workload-aware replacement, 避免仅根据 cache hit 选择策略。第三, 在固定质量集和多次 A/B 通过后, 再评估自适应 expert top- k 、PEU pruning 或与 EcoSpec 类成本模型结合的候选选择。第四, 在独立 Ascend 环境建立 CANN/torch_npu 或原生算子路径, 区分 CPU 页管理、主机到 HBM 搬运和 NPU 计算的各自瓶颈。第五, 以真实端侧模型完成 Agent Harness 开/关实验, 测量多轮任务的模型加载次数、上下文 token、TTFT、总 wall time 和工具等待占比。

总体而言, SLIM-ARC 的贡献不是一个固定的 madvise 参数, 而是一套把模型稀疏性、操作系统页面、存储行为和设备约束连接起来的运行时方法。其结果表明, 极限内存下的端侧 MoE 推理可以运行并获得显著设备内收益, 但最佳策略必须由访问阶段、内存压力和存储链路共同决定。

A 完整实验数据与复现信息

A.1 证据等级与比较规则

为同时保存研发过程和可用于结论的结果，本附录将实验划分为四类。A 类具有明确合同、重复或终态复核以及完整证据链；B 类来自真实运行，但样本较少或只验证单一机制；C 类存在链接错误、控制变量错误、跨缓存拼接或已被后续实验推翻，仅用于研发追溯；D 类证明构建、运行或硬件枚举，不代表目标加速路径。

所有 Prefill/Decode 均以 token/s 表示。提升率只在同设备、同模型、同缓存状态、同资源限制和同 workload 内计算。cold 与 warm、Q4_K_M 与 IQ4_XS、CPU-only 与 NPU、cgroup peak 与进程 RSS 均不直接相除。表中未给出方差的单次结果只用于趋势或机制解释。

A.2 模型与设备合同

表 14: 实验模型

模型	类型	量化/大小	用途
Qwen3-4B	Dense	Q4_K_M, 约 2.4 GB	小模型、KV/FA、Pi/RK 构建基线
OLMoE-1B-7B	MoE, 64 选 8	Q4_K_M 约 4.2 GB; Q2_K 2.56 GB	Router 链路 与 HiDevLab A/B
Qwen3-Next-80B-A3B	MoE, 512 experts	Q4_K_M, 48,410,988,384 B	决赛主模型
Qwen3-Next-80B-A3B	MoE	IQ4_XS, 约 40 GB	初赛量化探索, 非默认

表 15: 设备与主要瓶颈

设备	内存/隔离	存储	主要瓶颈
WSL/x86	8–32 GiB cgroup	NVMe	冷启动 I/O、缓存波动
RK3588	7.8 GB; 2.5–5 GiB scope	SSD 约 2.1 GB/s	ARM 算力、内存带宽、fault
Mac/Colima	1–4 GiB、swap 0	虚拟磁盘	page fault、CPU 饱和
Pi 5	4 GiB、运行期 no-swap	USB NTFS/FUSE	FUSE 往返与同步缺页
HiDevLab	aarch64; 32 GiB HBM 可枚举	CPU-only 实验	小模型接近全驻留

A.3 初赛 WSL/x86 数据

表 16: 80B WSL 历史核心结果

合同	配置	Prefill	Decode	解释
8 GiB, pp4/tg1	baseline	0.22	0.08	B 类 reference
8 GiB, pp4/tg1	SLIM-ARC	0.25	0.43	Decode +437.5%。仅限该合同
8 GiB, pp16/tg4	baseline	0.63	0.08	B 类 reference
8 GiB, pp16/tg4	SLIM-ARC	0.28	0.29	Prefill −56%、Decode +262.5%
16 GiB, pp4/tg1	SLIM-ARC	0.17	0.38	无同合同 baseline

历史实验中的 0.08 和 5.16 token/s 分别来自不同内存、缓存、量化和 FlashAttention 合同，不能组合计算统一端到端倍率；表16和表17因此分别保留各合同的原始结果。

表 17: 初赛量化、附属实验和精度检查

项目	结果	解释
IQ4_XS, 16 GiB/8 核	Decode 2.27 ± 0.38	tg64, B 类
IQ4_XS, 32 GiB/8 核	Prefill 4.44 ± 1.53, Decode 3.03 ± 0.29	B 类
IQ4_XS + FA auto, 32 GiB warm	12.99/5.16	不与 cold baseline 相除
Q4_K_M, 16 GiB	Prefill 1.96 ± 0.82	pp128, tg 超时
Q4_K_M, 32 GiB	Decode 2.68 ± 0.23	tg48
Qwen3-4B, 2 GiB+1 核	Decode 2.58 ± 0.17	小模型生存
OLMoE, 2 GiB+1 核	Decode 8.92 ± 0.94	小 MoE 生存
GSM8K 4B Q4_K_M	15/20	小样本检查
GSM8K 80B Q4_K_M + KV q4_0	5/10	小样本检查
GSM8K 80B IQ4_XS + KV q4_0	0/10	精度风险
Self-draft speculative decoding	3.01→1.41, -53.2%	拒绝

A.4 RK3588 完整实验数据

A.4.1 小模型与首次 80B 上机

表 18: RK3588 小模型与早期 80B 数据

模型/合同	配置	Prefill	Decode	备注
Qwen3-4B p64/n32/t4	F16 KV, FA auto	8.57 ± 0.01	6.90 ± 0.13	小模型参考
Qwen3-4B p64/n32/t4	KV q4_0	8.31 ± 0.10	5.05 ± 0.29	短上下文变慢
Qwen3-4B p64/n32/t4	FA off	7.63 ± 0.04	3.49 ± 0.69	FA 有效
OLMoE	默认	4.3	10.7	RSS 约 4.2 GB
80B p32/n16/t4	静态全开	0.39	0.23	负优化
80B p32/n16/t4	SLIM_ARC_DISABLE	2.02	0.89	reference
80B p32/n16/t4	NO_MADV_RANDOM	1.41	0.65	定位 RANDOM
80B p32/n16/t4	NO_PREFETCH	0.37	0.24	预取影响小
80B p32/n16/t8	静态全开	0.89	0.48	线程扩展
80B p32/n16/t2	静态全开	0.23	0.17	线程不足

80B llama-cli 冒烟中，静态全开/禁用/NO_MADV 的 Prompt/Generation 分别约 0.5/0.5、1.7/1.9 和 1.4/1.8，RSS 约 6.29/6.26/6.50 GiB。LD_PRELOAD 拦截确认静态路径对 45.1 GiB 映射调用 RANDOM 和 WILLNEED。

A.4.2 动态 MADV、长上下文与专家机制

表 19: RK3588 长上下文与动态策略

Workload	配置	Prefill	Decode	结论
p512/n64/t4	静态全开	2.43	0.43	负优化
p512/n64/t4	DISABLE	6.35	1.96	reference
p512/n128/t4	静态全开	2.41	0.53	负优化
p512/n128/t4	DISABLE	6.35	2.13	reference
context 8192/n128	静态全开	2.44	0.53	可运行
context 8192/n128	DISABLE	6.34	2.15	reference
context 16384/n128	静态全开	2.36	0.53	可运行
context 16384/n128	DISABLE	5.54	1.64	reference
p512/n128/t4	动态全开	6.09	2.08	接近 DISABLE

表 20: RK3588 专家预测与预算消融

配置	Issued(MB)	Hit rate	Decode	结论
baseline	27,836.7	31.05%	1.9	reference
confidence	12,123.6	55.35%	1.9	I/O -56%，速度无损
budget	25,848.9	28.78%	1.9	截断突发
confidence + budget	12,073.5	54.00%	2.1	探索性正结果
popular-16	72,634.4	19.31%	2.0	I/O +161%，拒绝
expert ON	—	—	1.41	p32/n16/t4
expert OFF	—	—	1.44	预取无收益

Temporal predictor 为 34.9%，spatial predictor 为 4.5%，随机参考约 2%。修复后 48 层注册 144 个专家张量，912 次 Router cache、846 次 cached expert 获取和 846 次 WILLNEED，证明专家调用链真实触发。

A.4.3 3 GiB、超长 Prompt、多轮对话与 RSS

表 21: RK3588 F/G/H/R 系列

编号	配置	MemMax	Prefill	Decode	结果
F1	baseline pp128/tg64	3 GiB	3.85	0.70	reference
F2	SLIM-ARC	3 GiB	4.40	2.21	tg +216%
F3	SLIM-ARC + KV eviction	3 GiB	4.19	2.12	tg +203%
F4	baseline	2.5 GiB	3.97	1.91	单次波动
F5	SLIM-ARC + KV eviction	2.5 GiB	4.26	2.21	tg +16%
G1/G2	4096 Prompt BL/SA+KV	4 GiB	—	—	600s Prefill time-out, 无 OOM
G3/G4	8192 Prompt BL/SA+KV	4 GiB	—	—	600s Prefill time-out, 无 OOM
H1	4B 五轮 baseline	3 GiB	7.88 avg	4.52 avg	输入有 -e artifact
H2	4B 五轮 SA+KV	3 GiB	7.86 avg	5.20 avg	Generation +15%
R3	3 GiB baseline RSS	3 GiB	4.29	1.26	VmHWM 4324 MB
R4	3 GiB SLIM-ARC RSS	3 GiB	4.57	1.31	VmHWM 3986 MB

编号	配置	MemMax	Prefill	Decode	结果
R5	3 GiB SA+KV RSS	3 GiB	4.57	1.33	VmHWM 3951 MB

R 系列使用 8 threads, 与 F 系列 4 threads 不直接比较。三个 3 GiB R 场景的 cgroup memory.current 均触及 3072 MB; 5.10 内核缺少 memory.peak, 使用 0.5 s 轮询峰值。

A.5 Mac/Colima 完整数据

表 22: Mac 8 月 12 日正式矩阵中位数

配置	Cache	Wall(s)	Decode	Major faults	决策
baseline	cold	69.530	0.600242	250,177.5	reference
control	cold	66.300	0.688606	406,601.5	reference
reclaim	cold	63.390	0.732030	403,548	opt-in, 调用 0
residency	cold	66.160	0.692031	403,203	opt-in
combined	cold	79.525	0.667540	399,309	rejected
baseline	warm	53.535	1.231783	143,727.5	reference
control	warm	60.300	0.820575	331,466.5	reference
reclaim	warm	58.080	0.844204	347,946	opt-in
residency	warm	63.190	0.712126	350,215.5	opt-in
combined	warm	63.265	0.759589	344,376.5	overall rejected

8 月 11 日生存扫描确认 2/3/4/6/8/12 GiB 均两次成功, 最低观察与最低稳定档位均为 2 GiB; CPU 曲线为 2c 75.06 s、4c 68.56 s、6c 66.28 s、8c 66.72 s。该镜像的 patched 行曾错误加载 baseline 库, 因此只保留真实 baseline、cgroup/no-swap 和 CPU 曲线。

表 23: Mac A1–A33 优化筛选

编号	优化	关键数据	决策
A1	latest-wins + pressure	2 GiB cold 3.951/0.800, 57.16 s; 1 GiB 成功	保留
A4	Prefill 8/Decode 6 threads	Decode +20.23%, wall −5.91%	候选
A5	expert RANDOM	Decode 0.801→0.262, wall 55.90→96.42	拒绝
A6	expert NORMAL	Decode 0.801→0.657, wall 55.90→61.08	拒绝
A7/A12	worker poll	50 优于 25/75/100	选 50
A8	shared expert residency	Decode +3.07%, wall −5.40%	opt-in
A9	MLOCK_ONFAULT	Decode −3.39%	拒绝
A10	Decode threads 5/6/7	6 threads 0.834、55.31 s	选 6
A11	readahead 128/192/256 KiB	256 KiB Prefill 4.732、51.77 s	Pareto
A13/A14	small always-used tensors	短 +0.11%; 长 −0.72%	不推广
A15	shared expert sustained	Decode 0.937→1.025, wall 101.87→96.36	保留
A16	256 KiB sustained readahead	pp +13.39%、tg +2.10%、wall −6.39%	候选
A17	graph 后 hot cache	Decode −2.07%	拒绝时机
A18	inline Router + hot512	tg +10.35%、wall −7.77%	保留
A20	layer-local 32 MiB	tg +1.03%、fault −4.09%、I/O −5.10%	opt-in
A22	cross-layer Router Top2	tg +9.96%、wall −7.77%、hit 88.56%	原型

编号	优化	关键数据	决策
A23	online transition Top4	三轮波动，无稳定优势	opt-in
A24	no expert prefetch	pp +10.66%、tg +22.45%、wall −15.72%	候选
A25	no entire prefetch runtime	tg −16.91%、wall +18.09%	拒绝
A32/A33	weight prefetch on/off	on 1.083/92.19 s; off 1.022/96.14 s	Mac 保留 on

8 月 17 日终态复核为 Prefill 3.908455、Decode 0.709095、wall 58.06 s、peak 2 GiB、Maximum RSS 2,084,740 KiB、major faults 402,118、expert samples 816、expert advice/issued/waste 0/0/0。

A.6 Raspberry Pi 5 完整数据

表 24: Pi 5 的 80B 慢存储序列

编号	改动	Prefill	Decode	Wall(s)	决策
A23	online transition Top4	0.200179	0.055052	550	原型
A24	no expert prefetch	0.201104	0.055614	547	慢盘参考
A26	resident hot512	0.200578	0.060726	522.5	保留
A28	shared + hot512	0.204023	0.063486	511	强基线
A29	16 MiB budget	0.204283	0.063585	510.5	I/O −98.44%
A32	no weight prefetch	0.203642	0.065246	505	Pi 候选
A33	weight prefetch control	0.203043	0.065006	505	无收益
A34	512 MiB LRU	0.203585	0.065856	503	保留
A35	384 MiB LRU	0.202685	0.064198	510	拒绝
A36	512 MiB LRU 复测	0.204150	0.066158	500	reference
A37	LFRU	0.200884	0.064845	507.5	拒绝
A38–40	二次命中准入	约 0.094	–	早停	Prefill 约 −54%
A41	NTFS/FUSE	0.204150	–	–	保留
A42	NTFS3 read-only	0.070032	–	–	拒绝
A43/A44	prefetch32/hot768	约 0.093	–	早停	控制变量错误，C 类
A45	exact file advice on	0.093601	–	早停	−0.2517%
A46	file advice off	0.093837	–	早停	reference
A47	旧源码复现	0.094405	–	早停	同一慢簇
A49	routed top-8	0.054285	0.029354	528	研究开关
A50	routed top-10	0.046178	0.026556	546	reference

Pi 的 Qwen3-4B 早期修复后 pp64/tg32 为 $10.16 \pm 0.23/3.48 \pm 0.13$ ，Peak RSS 约 2.42 GiB；该 Dense 模型不触发 MoE 专家路径，主要验证构建、mmap、KV 和 FA。

A.7 HiDevLab 与 Ascend 数据

表 25: HiDevLab OLMoE Q2_K 的 aarch64 CPU-only A/B

配置	Prefill	Decode	Wall(s)	Max RSS(KiB)
baseline	100.724597	51.340323	3.254600	2,575,956
patched-default	101.175889	45.254024	3.415535	2,577,888
patched-A24	100.602680	48.184150	3.373611	2,577,380
patched-no-prefetch	99.618235	45.803109	3.457525	2,577,572
patched-dynamic-normal	100.326690	47.396683	3.443797	2,577,576
patched-dynamic-random	100.237988	44.155583	3.535042	2,577,052
patched-sequential-prefetch	99.443328	47.358331	3.484104	2,578,344

硬件探测为 aarch64、Ascend 910B4、32 GiB HBM、Health OK、npu-smi 25.2.0、ascendhal 7.35.23。由于缺少完整 CANN/ACL/torch_npu，表25不使用 NPU。

A.8 跨设备优化决策

表 26: 优化在不同设备上的结果

优化	WSL	RK3588	Mac	Pi 5	总体决策
关闭 CPU repack	必需	必需	构建基线	必需	保留
mmap/demand paging	核心	80B 基础	2 GiB 基础	80B/4GB 基础	保留
静态 RANDOM	早期正向	4-6 倍负优化	可严重回退	慢盘不适合	拒绝默认
动态阶段建议	方向正确	恢复 2.84/1.40	保留	按慢盘收缩	设备化
盲目 expert prefetch	命中不稳	confidence 减 I/O	A24 关闭更快	关闭	不默认
Router predictor	研究	temporal 34.9%	A22 hit 88.56%	高命中未提速	研究
Shared/hot residency	未系统测	可选	A18 正向	A28/A34 正向	按设备保留
Wrong-page reclaim	未测	机制路径	正式矩阵调用 0	回收但 wall 中性	opt-in
Weight prefetch	混合	设备相关	on 更快	off 持平少 I/O	设备化
KV q4_0	8 GiB 有价值	小模型变慢	非主瓶颈	短上下文小	按压力启用
KV eviction	长上下文	8192/16384 可运行	非主项	短上下文不触发	长上下文 opt-in
FlashAttention IQ4_XS	热缓存收益 速度/体积收益	4B 明显 未作主线	后端决定 非固定模型	4B 可用 未用	兼容时保留 精度风险
Speculative decoding	-53.2%	未继续	未继续	未继续	拒绝

A.9 代码、数据与复现入口

核心运行时位于 `patches/llama-upstream/slim-arc-*`, 部署集成脚本为 `scripts/apply-slim-arc.py`。Mac 控制器和结构化证据工具位于 `scripts/macos/`; Agent Harness 位于 `scripts/agent/`。实验数据入口为 `docs/results/README.md`, 其下进一步链接 `docs/macos_test_notes/`、`docs/rk3588_test_notes/`、`docs/pi5_4GB_test_notes/` 和 `docs/ascend_test_notes/`。初赛报告保存在 `reports/Competition_Report/`, 本报告位于 `reports/Competition_Report_Finals/`。

A.10 赛事声明与成员分工

项目由欧阳易芃、马福泉和刘昊共同完成。主要工作包括系统架构与运行时实现、cgroup 与 benchmark 工具、跨设备部署、实验执行、数据分析、报告与演示材料。指导教师为赵帅和张献伟。开发过程中使用大语言模型辅助代码检索、文档整理和图表草拟; 所有进入报告的实验数值均来自仓库中的原始日志或结构化结果, 性能结论由队员复核。第三方代码、论文和系统接口均在正文与参考文献中注明。

参考文献

- [1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *International Conference on Learning Representations*, 2017.
- [2] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, pp. 1–39, 2022.
- [3] H. Du, S. Wu, A. Kharlamova, N. Guan, and C. J. Xue, “Flexinfer: Breaking memory constraint via flexible and efficient offloading for on-device LLM inference,” in *Proceedings of the 5th Workshop on Machine Learning and Systems*. ACM, 2025, pp. 56–65.
- [4] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, “Powerinfer-2: Fast large language model inference on a smartphone,” in *Proceedings of the European Conference on Computer Systems*, 2024, bibliographic entry checked against the locally archived manuscript.
- [5] H. Chen *et al.*, “KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models,” in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025.
- [6] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, 2022.
- [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [8] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *International Conference on Learning Representations*, 2024.
- [9] Linux Kernel Documentation, “Control Group v2,” <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>, 2026.
- [10] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” <https://github.com/ggml-org/llama.cpp>, 2024.
- [11] The Open Group, “posix_madvise: Memory advisory,” https://pubs.opengroup.org/onlinepubs/9699919799/functions/posix_madvise.html, 2018.
- [12] Qwen Team, “Qwen3-Next: Next-generation MoE language models,” <https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct>, 2025.
- [13] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” *arXiv preprint arXiv:2401.18079*, 2024.

- [14] Z. Liu, J. Ye, W. Li, Z. Yang, W. Xiong, L. Qiu, I. Lourentzou, T. Yu *et al.*, “KIVI: A tuning-free asymmetric KV cache quantization protocol for LLMs,” *arXiv preprint arXiv:2402.02750*, 2024.
- [15] Y. Song, Z. Mi, H. Xie, and H. Chen, “PowerInfer: Fast large language model serving with a consumer-grade GPU,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024.
- [16] X. Xia, J. Liu, X. Hou, P. Tang, M. Zhang, W. Wang, and C. Li, “Moe-prism: Disentangling monolithic experts for elastic MoE services via model-system co-designs,” *arXiv preprint*, 2025, locally archived manuscript; used only as design motivation.
- [17] X. Song, Z. Zhong, R. Chen, and H. Chen, “ProMoE: Fast MoE-based LLM serving using proactive caching,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.
- [18] J. Xie, R. Liu, H. Huang, Y. Ling, H. Dai, Y. Zheng, and W. Hu, “Less experts, faster decoding: Cost-aware speculative decoding for mixture-of-experts,” 2026.
- [19] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré *et al.*, “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Advances in Neural Information Processing Systems*, 2023.
- [20] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “SnapKV: LLM knows what you are looking for before generation,” *arXiv preprint arXiv:2404.14469*, 2024.
- [21] C. Xiao, P. Zhang, X. Han, G. Xiao, Y. Lin, Z. Zhang, Z. Liu, and M. Sun, “InfLLM: Training-free long-context extrapolation for LLMs with an efficient context memory,” in *Advances in Neural Information Processing Systems*, 2024.
- [22] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *International Conference on Learning Representations*, 2023.
- [23] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in *Proceedings of Machine Learning and Systems*, 2024.
- [24] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023.